

POCKET TRACKER

A Major Project Report Submitted in Partial Fulfillment for the Award of the
Degree of Bachelor of Technology in Computer Science and Engineering

Submitted

To



Dr. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY, LUCKNOW

Submitted by:

ADARSH SINGH (2200100100027)

ADARSH SINGH (2200100100025)

ADITYA SINGH (2200100100037)

UNDER THE SUPERVISION OF

Mrs. Gargi Tripathi

Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

UNITED COLLEGE OF ENGINEERING & RESEARCH,

PRAYAGRAJ

MAY 2026

CANDIDATE’S DECLARATION

We hereby declare that the project entitled “Pocket Tracker”, submitted by us in partial fulfillment of the requirements for the award of the degree of B.Tech. Computer Science and Engineering to Dr. A.P.J. Abdul Kalam Technical University, Lucknow, at United College of Engineering and Research, Prayagraj, is an authentic record of our own work carried out during the period from June 2025 to May 2026 under the guidance of Mrs. Gargi Tripathi, Assistant Professor, Department of Computer Science & Engineering.

We further declare that the matter presented in this project has not been submitted for the award of any other degree, diploma, fellowship, or any other similar title.

Signature of the Student

(ADARSH SINGH-2200100100027)

Signature of the Student

(ADARSH SINGH-2200100100025)

Signature of the Student

(ADITYA SINGH-2200100100037)

Place:

Date:

CERTIFICATE

This is to certify that the project titled “Pocket Tracker” is the bona fide work carried out by (Adarsh Singh), University Roll No. (2200100100027), (Adarsh Singh), University Roll No. (2200100100025), (Aditya Singh), University Roll No. (2200100100037) in partial fulfillment of the requirements for the award of the Bachelor of Technology (Computer

Science & Engineering) degree submitted to Dr. A.P.J. Abdul Kalam Technical University, Lucknow, at United College of Engineering and Research, Prayagraj.

The project is an authentic record of their own work carried out during the period from June 2025 to May 2026 under the guidance of Mrs. Gargi Tripathi, Assistant Professor, Department of Computer Science & Engineering.

The Major Project Viva-Voce Examination has been held on _____.

Signature of the Guide _____

[Mrs. Gargi Tripathi]

Signature of Project Coordinator _____

[Mr. Shyam Bahadur Verma]

Signature of the Head of Department _____

[Dr. Vijay Kumar Dwivedi]

Name & Signature of the External Examiner _____

[_____]

Place:

Date:

ACKNOWLEDGEMENT

We express our sincere gratitude to Dr. A.P.J. Abdul Kalam Technical University, Lucknow, for giving us the opportunity to undertake the Major Project during the final year of our B.Tech. (Computer Science & Engineering) program. This project has been an important and enriching learning experience in our engineering education.

We would like to extend our heartfelt thanks to Dr. Swapnil Srivastava, Principal, and Dr. Vijay Kumar Dwivedi, Dean Academics & Head, Department of Computer Science & Engineering, United College of Engineering and Research, Prayagraj, for their constant encouragement and support.

We also owe our deepest gratitude to Mrs. Gargi Tripathi for her valuable guidance, insightful suggestions, and constructive feedback throughout the course of this project, which has been instrumental in its successful completion.

We are thankful to all those who have directly or indirectly supported us during the project. Finally, we express our appreciation to the authors of the books, research papers, and online resources that we referred to while carrying out the project and preparing this report.

ABSTRACT

The Pocket Tracker Project is a Web Development Project. The project encompasses the design and development of a web-based Pocket Tracker application. Essentially, it was a comprehensive web full stack development project. More importantly, it was designed to help me learn full stack web development. The application can act as a personal pocket money management system application. It was also designed and developed to enhance my skills for building a fully integrated web system. The complete application is developed using the MERN stack. MERN stack basically refers to a full stack JavaScript model with a modern framework that is robust.

The client and server interaction was made possible by the simultaneous development of the front-end and back-end components. Also, continuous testing was conducted in the development phase to eliminate bugs during the course of the process.

On completion of this project, the systems developed will have an easy to use interface to enter, view and manage transaction. In addition, a bar chart is added to show monthly payments. A user authentication module is also implemented to secure user data and transactions apart from this.

All in all, the project successfully achieves the aim satisfactorily. The comprehensive pocket tracker works perfectly, and it looks easy to use.

TABLE OF CONTENTS

Title Page	
Declaration of the Student (Signed by Student)	i
Certificate of the Guide (Signed by Guide, HoD, Project Coordinator)	ii
Acknowledgement	iii
Abstract	iv
Table of Contents	v
List of Figures	vi
Chapter 1: Introduction	1
1.1 Problem Definition	1
1.2 Project Overview/Specifications	3
1.3 Hardware Specification	6
1.4 Software Specification	7
Chapter 2: Literature Review	11
2.1 Existing System	11
2.2 Proposed System	12
2.3 Feasibility Study	13
Chapter 3: Methodology Used	20
3.1 Requirement Specification	20
3.2 Flowcharts / DFDs / ERDs	22
3.3 Design and Test Steps / Criteria	24
3.4 Algorithms	31
3.5 Testing Process (Test Cases to be included)	35
Chapter 4: Results and Discussion	37
Chapter 5: Conclusion and Recommendations	43
Chapter 6: Appendices	54
References	58

LIST OF FIGURES

Figure 3.2.2.1 Level 0 DFD	22
Figure 3.2.2.2 Level 1 DFD	23
Figure 3.2.3 ER Diagram	24
Figure 3.3.1.1 Database Schema	25
Figure 3.3.3.1 Class Diagram	27
Figure 3.3.3.2 Sequence Diagram	28
Figure 3.3.3.3 Activity Diagram	29
Figure 3.3.3.4 Input Output Diagram	30
Figure 3.4.2 Integration Testing	34
Figure 4.1.1 User Registration	37
Figure 4.1.2 User Login	38
Figure 4.2.1 Add Transaction	39
Figure 4.2.2 View Transaction	40
Figure 4.3.1 Dashboard Visualisation	41

CHAPTER 1

INTRODUCTION

1.1 PROBLEM DEFINITION

1.1.1 Few Customization Features Problem:

Most of the apps have preset categories and formats.

Explanation: The users might have some unique spending or earning patterns that will not be covered by the pre-designed categories and thus affect the efficiency of using the application.

1.1.1.1 Bad UI/UX Problem:

There are applications whose interface is crowded and navigation process complicated.

Explanation: Such a problem might lead to the users being unable to record their transactions in an efficient manner.

1.1.1.2 No Real-Time Synchronization Problem:

Sometimes the changes in one device might not be reflected immediately on other devices.

Explanation: Such an issue might be common for people who use the application across different devices like a phone and computer.

1.1.2 Insufficient Data Visualization Problem:

Most platforms provide basic numerical data without any charts or graphs.

Explanation: It is challenging for users to analyze their spending patterns or budget without visualization features.

1.1.2.1 Insufficiency of Currency and Localization Support Problem:

Problem: Pocket Tracker applications may lack support for various currencies and localization capabilities.

Explanation: Such a situation would significantly reduce the functionality of the software, which will limit users from operating in a multilingual environment.

1.1.2.2 Lack of Proper Security Features

Problem: Not all platforms incorporate the required data security measures.

Explanation: The information provided on such a platform will be highly confidential; hence, users will have problems regarding data breaches or leaks due to the lack of security features.

1.1.3 Failure to Integrate Banking and Wallet Services

Problem: Certain applications do not provide services with financial institutions or banks.

Explanation: Users will have to input information about their financial activities manually, which might lead to errors during the process.

1.1.3.1 Unavailability of Offline Features Problem: Most applications require an active internet connection to operate correctly.

Explanation: This could pose significant challenges to travelers or those who live in poorly connected areas since they will be unable to record any transactions instantly.

1.1.3.2 No Budget Setting Option

Problem: Not all expense tracker applications allow users to plan budgets for each month.

Explanation: Such a limitation would deny users from making future financial plans.

1.1.4 Insufficient Customer Support and Documentation

Problem: There are instances where users experience bugs in the application, and they need assistance that is not easily available.

Explanation: Insufficient customer support and/or documentation will frustrate users and make them leave the application.

1.2 Overview / Specifications of Project:

1.2.1 Project Title – Pocket Tracker

The project proposal mentioned the full stack web application which helps the user track the personal expenses in an arranged and aesthetic way. The MERN stack served as a basis for programming this personal expense manager application. You could write the expenses, budget, categorize, graphically, and many other operations it supports. The goal of this application is to create financial awareness as well as easy manage finances.

1.2.1.1 Technology Stack - MERN (MongoDB, Express, React, Node.js)

The application uses only JavaScript technologies as a technological stack. In this case, the NoSQL database that is utilized is MongoDB, the web framework is Express.js, the React library is for front-end development and Node.js runtime platform is for carrying out the back-end activities.

1.2.2 User Authentication and Authorization

The application offers features that enable users to securely register their account. Additionally, users will be able to log in. We will use JWT tokens for authentication and crypt hashes for additional security of users' passwords.

1.2.2.1 Responsive User Interface (UI)

The user interface, or what you see in the application, is made with React.js and gets its look and feel from things like Bootstrap or Material UI. Importantly, the

interface adjusts to the size of the screen, meaning you can deal with your spending on a desktop computer, a tablet, or a phone. It's easy to get around and everything is laid out in the same way, which makes it easy to use and gives you a fast, up-to-date experience.

1.2.2.2 Expense Management Module

The Expense Management Module is the main part of the app, and is where you'll keep tabs on your money. You can add each purchase or bill by typing in how much it was, what it was for, what kind of expense it is, and when it happened. You're able to change or remove things you've already entered, and it's this section that lets you follow your spending and have a good, correct history of where your money is being spent.

1.2.2.3 Custom Expense Categories

You can use expense categories that are already set up in the application, or you can make your own for a system that's tailored to you. Because of this adaptability, you're able to classify your expenses in a way that really matches how you manage money - so you could distinguish between groceries and meals at restaurants, or break apart your utilities and rental payments.

1.2.2.4 Data Visualization and Analytics

The app has charts and graphs (using Chart.js or Recharts) to help you understand your financial situation better. These visuals, for example bar charts and pie charts, give you a simple way to look at your spending, by category and how it changes as time goes on, to help you make smarter and more instinctive financial choices.

1.2.3 Monthly Budget Planner

With the monthly budget planner, you decide how much you can spend for each month. Then, the system keeps an eye on what you are spending compared to that amount and will tell you if you're going over. This helps you be sensible with your money, and keeps you from spending too much, which means you're more likely to save or achieve whatever financial aims you have.

1.2.3.1 Expense Filtering and Sorting

You can find expenses by what they're for (the category), when they happened (a certain time period), or how much they cost. And, once you've found them, you can put them in order. This gives you much more control, lets you quickly locate particular transactions, and makes the app useful for going over your finances regularly or for a proper audit.

1.2.3.2 RESTful API Architecture

The way the application works with all its information is through a RESTful API. It's created with Node.js and Express.js and this structure makes the application broken down into separate, parts, easy to get bigger, and simple to update. The specific 'places' to send requests to in the API (endpoints) deal with things like logging in, payment details, spending types, and financial plans, and they do it both safely and without wasting time.

1.2.3.3 MongoDB Database Schema

MongoDB is where we keep all the information, and it stores everything in a really bendy format a bit like JSON. We use 'collections' within MongoDB to hold details on users, what they've spent, the different types of spending, and their budgets. Because it doesn't need a strict format (it's 'schema-less') it's very quick to build and change things as we add new parts to the application, and it's simple to make it handle more and more data.

1.2.4 Form Validation and Error Handling

We check what users type in using both the part of the application you see (with React Hook Form or Formik) and the part on our computers (with Joi or some code we've written ourselves). Having these two checks is to make sure bad information doesn't get into the database, which makes everything more reliable, and gives you nice, easy-to-understand error messages.

1.2.4.1 JWT-Based Session Management

How we keep track of your login (sessions) is with secure JWT tokens. These are saved in your browser's local or session memory. They don't last forever though; after a certain amount of time they become invalid and you'll be logged out if you haven't been active. This makes the application more secure, gives you better control over your login, and stops people getting into your account or misusing your token.

1.2.4.2 Cloud Hosting and Deployment

For the part of the system users actually see (the 'front end') we're using Vercel or Netlify. Render or Heroku take care of the 'back end' – all the processing that happens behind the scenes. We're keeping the database in the cloud with MongoDB Atlas. This way, the system is almost always on, updating is simple, and it can handle lots more people or new features as it gets bigger.

1.2.4.3 Real-Time Notifications and Feedback

You get immediate confirmation when you do things with your expenses - when you add one, change one, or get rid of one. This happens through little pop-up messages (toast notifications or alerts) that tell you if it worked or didn't, making everything flow better and the app feel more responsive.

1.3 Hardware Specification:

For building, you'll need a development machine with these specifications. It needs at least an Intel i5 (or something better) to quickly handle backend tasks and compile your project, and a minimum of 8GB of RAM for comfortably running all your development applications at once.

You'll also want 256GB of SSD storage, or more, because databases and your code will load and save much faster. An integrated graphics card is enough, you don't need a super powerful one.

You can use Windows 10 or 11, macOS or Linux, as all of these work with Node.js and MongoDB. A good, reliable broadband internet connection is essential for installing with npm, testing your application with APIs, and sending it out for use.

Finally, a good cooling system is important to prevent overheating during lengthy builds and releases.

1.4 Software Specification

1.4.1 Development Tools

For development, we'll be using a collection of programs. Git is the system for tracking changes to the code. With it we have GitHub, which is where the code will be stored remotely and used to work with others. For finding errors in the browser, Chrome or Firefox's built in developer tools will be useful.

Development itself will involve building the user interface. React.js will be used for the web application's front end as it's a library for creating lively, changing interfaces.

Express.js is the web framework to build the back end application programming interface (APIs), and MongoDB is a NoSQL database to hold the information about expenses. Node.js is the environment for running JavaScript on the server.

Finally, hosting and deployment covers how people will actually use the app. To easily host the website, we'll use either Vercel or Netlify for the front end.

1.4.2 Programming Languages and Libraries

JavaScript is the main language for the whole MERN stack. It's what makes the part of the website you actually see (React) and the behind the scenes part (Node.js) work. It lets things happen on your screen without a page reload, and handles things that don't happen in order, like requesting information from an application. Plus, it has newer versions of JavaScript (ES6 and later) with useful tools like shorthand function definitions, a way of breaking data into easier to use pieces and a simpler

way to manage tasks that take time, all of which makes your code easier to read and faster.

- JavaScript lets you change things on a webpage as things happen, showing campaign news, how transactions are going, and how much backers have given, all as it's going on.
- It also manages what happens when people using the site do things; for example, clicking things, sending off forms, or moving around on the site will start the platform doing something, perhaps starting a blockchain deal or sending campaign info.
- JavaScript also lets the site talk to its 'behind the scenes' part in the background, getting info without you having to refresh the page. Getting current campaign details, the total in donations and similar information in the moment is important and this is how it's done. Because JavaScript is so adaptable, you can quickly make changes and try things out which is really important when the platform is first being built and tested.

1.4.2.1.1 HTML5

HTML is the fundamental building block for organizing how a user interface looks. With it, you create meaningful parts of a page - things like forms, buttons, and places to type (inputs). If your site is available to everyone on the internet, using HTML properly also helps people using assistive technologies and search engines find what they need. And in the expense dashboard, it's used alongside React components to decide exactly where everything appears on the screen.

1.4.2.1.2 CSS3

It makes HTML and React components look good. You can create designs that adjust to any screen size with Flexbox, Grid, and media queries. You can use things like Tailwind (a framework) or styled components (a library) to keep your styling organized. All of this is really important for making a user interface that people will enjoy using and that looks great.

1.4.2.2 React.js

You can use this to create UIs that are made of parts which change and respond to the user. It keeps track of how things are changing (like your spending, and when it's updated) with “hooks”—specifically `useState` and `useEffect`. React Router is the part that handles having many pages, for things like logging in, the main dashboard view, and your analytics. It also handles forms, pop-up boxes, and makes the list of transactions update as they happen.

1.4.2.2.1 Node.js

Essentially, this is where JavaScript can operate on the server. It's in charge of the important parts of your application, such as confirming transactions, looking after who is logged in (authentication), and the usual creating, reading, updating, and deleting of data. To deal with lots of requests at once and stay quick, it doesn't wait for one thing to finish before starting the next (non-blocking I/O). Plus, it works nicely with Express, making it simple to create RESTful APIs.

1.4.2.2.2 Express.js

It's a really easy-to-use structure for building things with Node.js on the web. You can more easily organize which URLs do what (routing), add extra steps to handle requests (middleware), and deal with mistakes (error handling). It's meant to help you create APIs for doing things at specific web addresses—things like adding an expense, deleting something, registering for an account, and so on. It also looks after who's using the system (authentication), checks the data people type in (input validation), and manages their use of the site over time (session management).

1.4.2.3 MongoDB

This NoSQL database keeps your expense information, user details and categories as document files similar to JSON. It works smoothly with Mongoose, so you can check your data is correct and ask questions of it. Because it's very fast at both getting and saving information, it's great for keeping up with lots of transactions

happening all the time. It also allows you to have items inside items (nested objects) which is useful for showing each person's own record of what they've spent.

1.4.2.3.4 Mongoose

This is a MongoDB library for organizing your data (called DOM). It lets you make sure your data is properly shaped and creates models for it. It makes the basic things you do with data - creating, reading, updating, deleting - much easier. For instance, you can save a new expense or find past expenses by someone's user ID. And it includes "middleware" meaning you can run code before something is saved, or check it, to enforce rules (like a maximum amount for a transaction).

1.4.2.3.5 Axios / Fetch API

When your website or app (the 'frontend') needs information from the main application or server (the 'backend'), you'll use something to make those requests. Axios is one of these things and it makes it much easier to get data (with GET requests), send new data (POST requests), update data (PUT requests), or remove data (DELETE requests) from the backend's API. You can use it for sending off forms, getting details of what someone has bought, or letting people sign in. Axios also lets you add security information to each request, using something called 'interceptors' for authorization headers.

CHAPTER 2

LITERATURE SURVEY

Pocket Tracker Application - Pocket Tracker is a strong, easy-to-use program for Android that lets you record all your daily spending in a digital diary. Because it breaks down what you've spent into different types of purchases, you can get a clearer idea of where your money is going and find it easier to stay within your budget. For a really good look at how you spend, the app gives you monthly reports which are charts of your costs, and it groups these costs by type. Plus, Pocket Tracker keeps a record of everything you've spent before, so you can look back at past expenses whenever you want. In short, this uncomplicated app will help you monitor what you're spending, be better with your finances, and make smarter choices about money.

2.1 Existing System

2.1.1 Handwritten Logs

People can write down what they spend in actual notebooks or diaries. It's a straightforward and doesn't cost anything, but you can't get it to do calculations for you or give you a deeper understanding of your spending.

2.1.2 Spreadsheets

With programs like Microsoft Excel or Google Sheets, you can do fairly simple tracking in columns and rows, use formulas, and make charts. But you have to type everything in yourself, and it's easy to make mistakes; plus there isn't much happening automatically, and it doesn't really connect to anything else.

2.1.3 Standalone Mobile Applications

Some people use apps on their phones for pocket tracking. These are handy, but often they aren't connected to your other finances, so they don't update with your bank account or work on the internet.

- **Limitations of the Existing System Lack of Integration:** The current systems don't work together. Mobile apps and spreadsheets, for instance, are separate, and this breaks up your financial information.
- **No Real-Time Synchronization:** Changes you make in one place aren't immediately visible on other platforms or on your phone.
- **Limited Analytics:** When it comes to understanding your money, older ways of doing things don't offer much in the way of useful analysis, meaning you get hardly any monthly overviews or predictions of future spending.
- **Error-Prone Manual Methods:** Because data is entered by hand, it's very easy to get numbers, dates or what things are for mixed up.
- **Inadequate Customization:** It's difficult for individuals to adjust the system to do things their way, like setting up budget warnings or labels.
- **Limited Security:** And finally, keeping data in a notebook or a simple app isn't very safe from being lost, stolen or seen by people who shouldn't have it.

2.2 Proposed System

This system is a current web app for tracking expenses and it's been made with MongoDB, Express.js, React.js and Node.js (that's the MERN stack!). It aims to be smooth to use, safe, and clever in how it helps you handle and look at spending for yourself or a business.

The main parts of the system are as follows.

- For logging in and signing up, the system will be secure, using JWT or OAuth. Access to different things will be controlled by what 'type' of user you are (so an admin will have different access to the system than a regular user, for example).

- You'll be able to add expenses, change expenses, remove expenses, and put them into types like food, bills, or fun. You can also quickly find expenses by filtering and searching.
- Changes to data on the client and server are kept in step, either with web sockets or normal RESTful APIs, and data is the same no matter what device you're using.
- You can see your data visually, with pie charts, bar graphs and how things are changing over time, and there are tools to help with budgeting and predicting future spending.
- The system is much more secure, because data is encrypted when it's being sent (HTTPS), securely stored, and uses current ways of confirming who you are.
- Because of the way the database works, any changes show up immediately, so you're always looking at the right and most recent information.
- Finally, using React.js and CSS frameworks like Tailwind CSS or Bootstrap, the system is designed to be good to use, respond to your screen size and be easily understood, with a visual interactive element.

2.3 Feasibility Study

Before starting a project, it's absolutely necessary to do a feasibility study – to figure out if it's actually doable and likely to work, and to look at that question in lots of different ways. We'll start by looking at the technical side of getting the Pocket Tracker platform up and running. A thorough feasibility study gives everyone with a stake in the project a good sense of what it could achieve, and it's what you'll return to as a core guide for making choices as the project is built.

2.3.1 Technical Feasibility

Basically, technical feasibility asks if we have the technology, the existing systems, and the expertise to do what we're planning. That means looking at the hardware, software, tools, and platforms we'd use to build, launch and keep running the application.

- Pocket Tracker is created with the MERN stack: MongoDB, Express.js, React.js, and Node.js. MongoDB is the database. It's a NoSQL database for storing data in a flexible, JSON-like format and is easily expanded horizontally, which is great for data that changes quickly or isn't in a standard structure. MongoDB Atlas is a cloud based, managed MongoDB and they have a free option for getting started.
- Express.js is a simple web framework for Node.js which makes the server-side logic, how the application routes requests, and managing middleware easier. It's flexible and simple, allowing for quick development and being able to handle lots of users later on. React.js, from Meta, is a library for building interactive interfaces from parts called components. It is very efficient with managing how things change and with its virtual DOM, makes the front end of the application fast and responsive. Node.js is what allows JavaScript to be used on the server. It is built to handle many inputs and outputs at the same time, which is typical for today's web applications.
- This combination of technologies is popular, constantly updated, and has a huge amount of tools, libraries and online groups to help developers. Using it makes developing quickly, being able to grow the application, and maintaining it in the future all easier. Because JavaScript is used throughout the entire stack, development is more consistent, teams work together more easily, and the development process is simpler.
- Getting a software project running smoothly, without issues, and with as little downtime as possible is really important for deployment. Pocket Tracker has excellent support for deploying to cloud-based and version controlled environments. You can deploy the application to cloud providers like Heroku, Vercel, or Amazon Web Services (AWS). These have different prices, can grow with your needs, and include ways to keep an eye on how the application is doing. Heroku and Vercel are good for quickly getting something live and continuously updating it, while AWS is better for very large applications that need to be available everywhere.

Continuous Integration and Continuous Deployment (CI/CD) is supported by GitHub Actions, Travis CI, or Jenkins. These tools automatically build, test and deploy the application, meaning you can make changes and get them live quickly with less manual work.

- GitHub is used for version control and fits into the development process perfectly. It allows developers to work together using things like 'pull requests', reviewing each other's code, and tracking problems. This helps people communicate and make the code better. Also, Docker can be used to make deployment even easier and to make sure the application behaves the same in development, testing and when it's live. Because of all this, the Pocket Tracker project's technical base isn't only possible but designed for modern software development and how applications are deployed.

2.3.2 Economic Feasibility

- When looking at any tech project, working out if it's economically sensible is really important. It helps everyone involved understand if the money spent on creating, starting and continuing it will eventually give good results. The Pocket Tracker project has been carefully put together to be financially realistic, particularly when you think about who it's for and how many people are likely to use it. It uses development methods which don't cost a lot, and depends on tools and platforms available for free, so you don't need a huge amount of money at the start or for running it.
- To keep the development costs as low as possible, the project uses a strong and effective set of technologies that are all free and open source. MongoDB (the database) uses MongoDB Atlas, which has a generous free option for smaller to medium apps. This cloud database is flexible, dependable, performs well and doesn't have any initial license costs. For the 'backend' (what makes the app work behind the scenes), it uses Node.js and Express.js - also free and open source, and with a lot of support from other developers. These make building the server side of things quicker and without having to pay for platforms. React.js, a Javascript library from Meta and also free and open source, is used

for the user interface. React helps build interfaces that are in sections, easy to update and work quickly, without needing to buy expensive UI sets or programs that aren't open. This approach really brings down the cost of software licenses and makes sure the project can easily change in the future, particularly as it gets more popular.

- A major benefit of using Javascript for everything in the app is that it lowers the total cost of development and the people working on it. Because Javascript is used for both the front and back end, things are simpler and it's easier to find and train developers, as they only need to be good at one language to work on all parts of the code. And all the technologies chosen have a large, active community of developers around the world. This means you don't need expensive official training or advisors, because most problems can be solved using online forums, documentation, or guides created by the community. These things make development much faster and cut down on extra expenses.
- Another financial plus of the system is how easily it can grow. It's designed to start using the free levels of service from cloud providers, and then move to paid options as it becomes more popular - and you only pay for those as you actually need them because of more users or needing to store more information. As the whole system is on the cloud, there's no need to initially spend money on physical servers or a complicated, expensive infrastructure. This lowers the initial outlay of money and is perfect for a 'lean startup'. All these choices mean the system will stay financially stable and can deal with growth.

2.3.3 Operational Feasibility

- How easily the system will work in the setting it's meant to be in, and how well it fits with what people need and the aims of the organization, is what we mean by 'operational feasibility'. A system could be great from a technical point of view, but if it's awkward to use, it won't be used by many. Pocket Tracker really prioritizes being easy to operate, a smooth experience for the user, and features that adapt to the way you work.
- User-Friendliness is a huge part of making this happen.

The app has a current, adaptable user interface (UI) built with React.js. It's visually tidy, makes sense to navigate, and is easy for anyone to use, even if they don't know a lot

about computers, and it lets them do things efficiently. The important things it does are done in ways most people will already understand, so you won't have to spend long learning it. Help that pops up when you need it, explanations when you hover over things and visual clues all help you understand and use it with more ease, and so you will use it more. In fact, the interface is designed to help people be satisfied with it and user satisfaction is essential for it to be useful.

- Customization and Control is the next point, and the system gives people a lot of options to suit them. You can make your own expense categories, so the tool fits with your own way of budgeting and helps you keep a closer track of your finances and plan towards your objectives. You can also set up dashboards and reports to show the information and figures that you want to see. So, whether you are looking at what you're spending, how things are changing, or how you are doing against your goals, the system provides relevant details and gives you control. Being able to personalize it like this makes it more useful and encourages you to use it regularly.
- Finally, Accessibility and Responsiveness* are important for people to use it and to be pleased with it. It works without a hitch on computers, tablets and phones, and being available on all devices is very important for people who need to get at their finances wherever they are. The system uses data that is updated without you having to do anything and it shows you changes in realtime. Any changes you make show up immediately in the app, building your confidence and letting you make quick choices. Because it is both easy to get to and reacts quickly, people will use it more and it will be easier to reach the objectives it is meant to help with.

2.3.4 Legal Feasibility

- The application is built to be legally sound, following all relevant laws, guidelines and rules - especially those regarding safeguarding user data, digital communication and rules for each area where it is used. The Pocket Tracker project is being designed with the law in mind to avoid problems with being in or out of compliance.

2.3.4.1 Data Privacy Compliance

- To be legally above board and keep people trusting us, the project uses the very best ways to deal with data.
- Regarding User Data Handling: All of a person's private and financial details are encrypted while being sent and whilst being stored. This means we're using strong encryption, and hiding sensitive information within data.
- For Compliance with Local Laws, the system will fit with important privacy rules all over the world and in specific regions, like:
- GDPR (for people in the EU): This gives people control over their information and makes sure data is handled legally.
- CCPA (for people in California): This means we're open about what we do with data and give people the option to not have their data used.
- IT Act 2000 (for India): This is in line with India's internet and data protection laws and makes sure online transactions are safe.
- Because we are making sure we comply with the law as we build it, the application shouldn't have legal issues later.

2.3.4.2 Secure Data Transmission

- It's vital that data is sent between users and the server safely.
- All communication uses HTTPS with SSL/TLS encryption. This stops anyone intercepting, changing, or inserting themselves into the data flow.
- We continuously check for weaknesses and monitor things, to make sure the platform is protected from new threats. These steps meet our legal obligations and keep users' faith in us.

2.3.4.3 User Consent and Transparency

Being open and honest is at the heart of being legally compliant and running things ethically.

- Explicit User Consent: Before anyone can use the application, they must agree to the Terms of Service and Privacy Policy. That way they are using it knowing what they are agreeing to.

- **Clear Data Policies:** We tell users exactly what information we gather, why we need it, where we keep it, and when we get rid of it. They also have ways to control their own information, including deleting their account and getting a copy of their data.
- **Global Legal Alignment:** As Pocket Tracker might have users from all over the world, the application is arranged to comply with rules about crowdfunding, digital payments and cryptocurrencies where needed. Being legally flexible means it will continue to work in different countries.

CHAPTER 3

SYSTEM ANALYSIS & DESIGN

3.1 Requirement Specification

3.1.1 Functional Requirements

3.1.1.1 User registration and authentication:

Secure login system.

JWT – based session management.

3.1.1.2 Project creation and management:

Create new project (name, description, start/end date).

Edit and delete project.

3.1.1.3 Expense tracking:

Enable users to add daily expenses (amount, date, category, note, project).

Categorize expenses (e.g., travel, material, labor).

3.1.1.4 Expense Reports:

View total and categorized expenses per project.

Graphical summaries (bar/line charts).

3.1.2 Non-Functional Requirements

3.1.2.1 Security:

Protect user data and funds against threats.

HTTPS communication.

3.1.2.2 Usability:

Ensure the platform is easy and intuitive to use.

Responsive UI for desktop and mobile.

3.1.2.3 Performance:

Fast response times even during heavy usage.

Faster page loading. **3.1.3**

Hardware Requirements

3.1.3.1 Server requirements:

Reliable cloud servers or decentralized nodes.

3.1.3.2 User-end device requirements:

Devices supporting modern web browsers and developing frameworks.

3.1.4 Software Requirements

3.1.4.1 Development tools:

Tools like Node.js, and React.

3.1.4.2 Web development platforms:

MERN or similar for developing responsive web application.

3.2 Flowcharts / DFDs / ERDs

3.2.1 Flowcharts

3.2.1.1 User journey flow:

Steps from signup to expense tracking.

3.2.1.2 Admin/project owner flow:

Process from income/expense submission to expense tracking.

3.2.2 Data Flow Diagrams (DFD):

3.2.2.1 Level 0 DFD:

Top-level system overview.

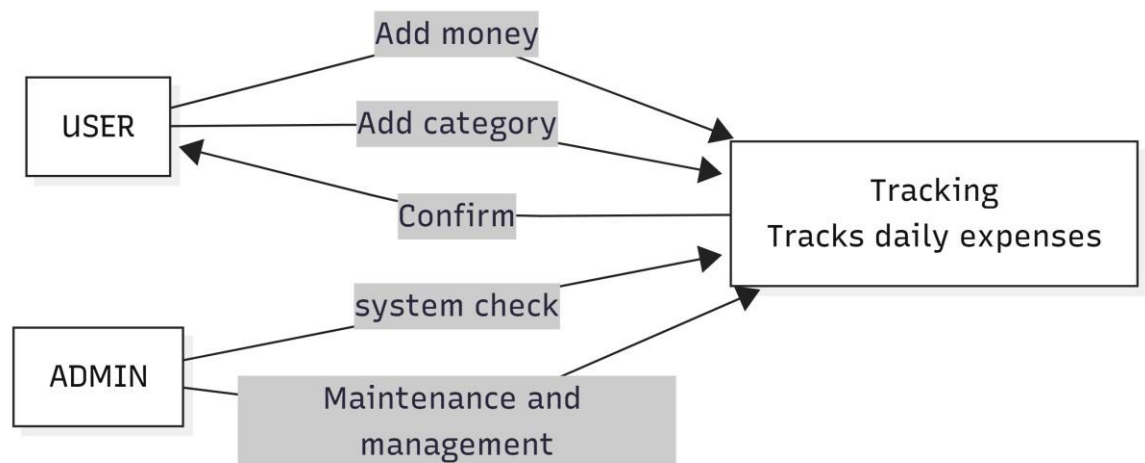


Figure 3.2.2.1 Level 0 DFD

The above diagram shows the DFD level-0 where user adds money to the tracking system. The tracking system updates it to the database then data base will retrieve it to tracking system. After retrieving the system confirms data to the user. The admin checks system and maintains the system.

3.2.2.2 Level 1 DFD:

Detailed internal data processing.

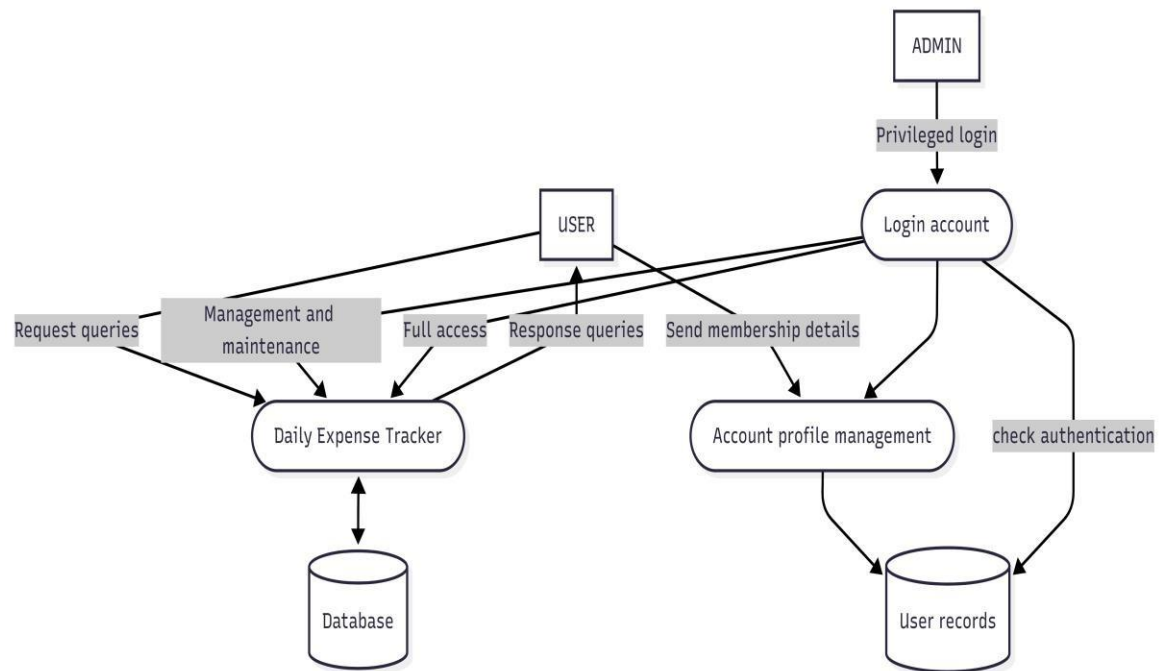


Figure 3.2.2.2 Level 1 DFD

The above diagram is for DFD level-1, where we can track data through the processes it is involving. The process daily expense tracking is further divided as account profile management and login account management. The user requests queries to the daily expense tracker and gets response from it. The login account checks the authentication of the user records. The account profile management sends membership details to the user.

3.2.3 Entity-Relationship Diagrams (ERD)

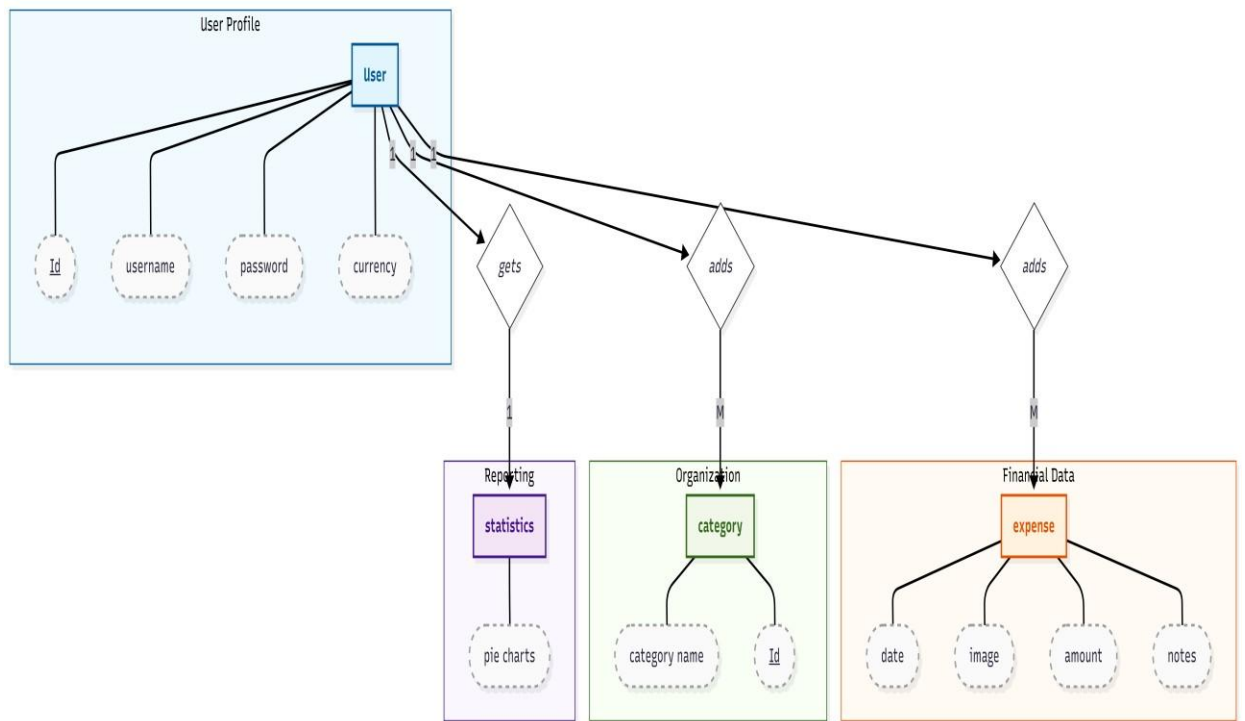


Figure 3.2.3 Entity-Relationship Diagrams (ERD)

The above diagram explains the relationship between the databases where rectangle represents entity, oval represents attributes and diamond represents relation. There are four entities with their respective attributes.

3.3 Design and Test Steps / Criteria

3.3.1 System Design Principles

Systems design is the process of defining the architecture, modules, interfaces, and data for a system to satisfy specified requirements. Systems design could be seen as the application of systems theory to product development.

3.3.1.1 Database Schema

There are five tables in our application database which are user, expense, result, income and daily list. In the above diagram, the tables cover their respective primary key and their fields.

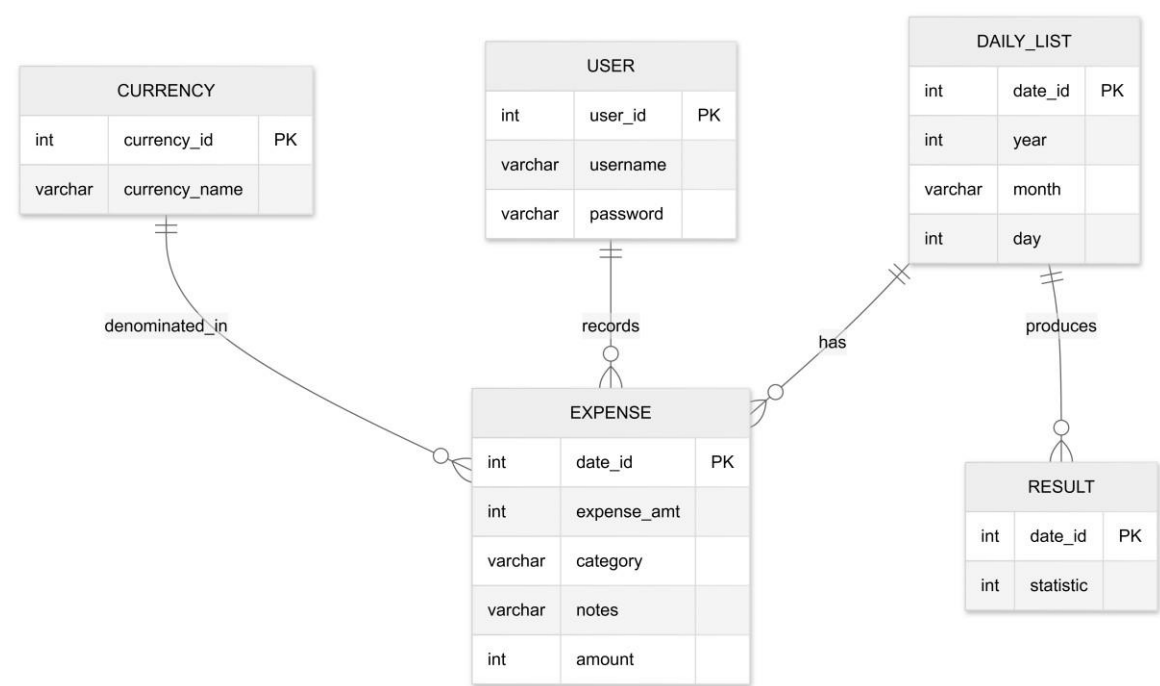


Figure 3.3.1.1 Database Schema

3.3.2 Data Dictionary

Essentially, a data dictionary (sometimes called a metadata repository) is a key part of planning and explaining a database and the whole system around it. Think of it as a single, thorough listing of all the data in your information system with a lot of detail. This covers what each ‘thing’ (or entity) is, what each of its qualities (attributes) are, what sort of information those qualities hold (data types), how these ‘things’ connect to each other (relationships), what restrictions exist on the data (constraints), where the data began (origin), and how it’s actually used. Developers, analysts and people using the system can all use the data dictionary as a guide to make sure everyone understands the data in the same way and uses it correctly throughout the entire project.

Keeping a data dictionary up to date helps manage your data better, makes the system simpler to update and repair, and helps separate parts of the system or different people involved with the database work together. It’s also very important for data being accurate and correct, by making certain the data is in the correct format and the relationships between things are as they should be.

S. No.	Entity	Attribute	Data Type	Constraints
1.	User	user_id	int	Primary Key
		username	varchar(20)	Not Null, Unique
		password	varchar(20)	Not Null
2.	Daily_List	date_id	int	Primary Key
		year	int	Not Null
		month	int	Not Null
		day	int	Not Null
3.	Expense	date_id	int	Foreign Key (Daily_List)
		exp_id	int	Primary Key

		expense_category	varchar(20)	Not Null
		expense_notes	varchar(20)	Optional
4.	Result	statistics	long int	Not Null

3.3.3 UML Diagram •

3.3.3.1 Class

Diagram

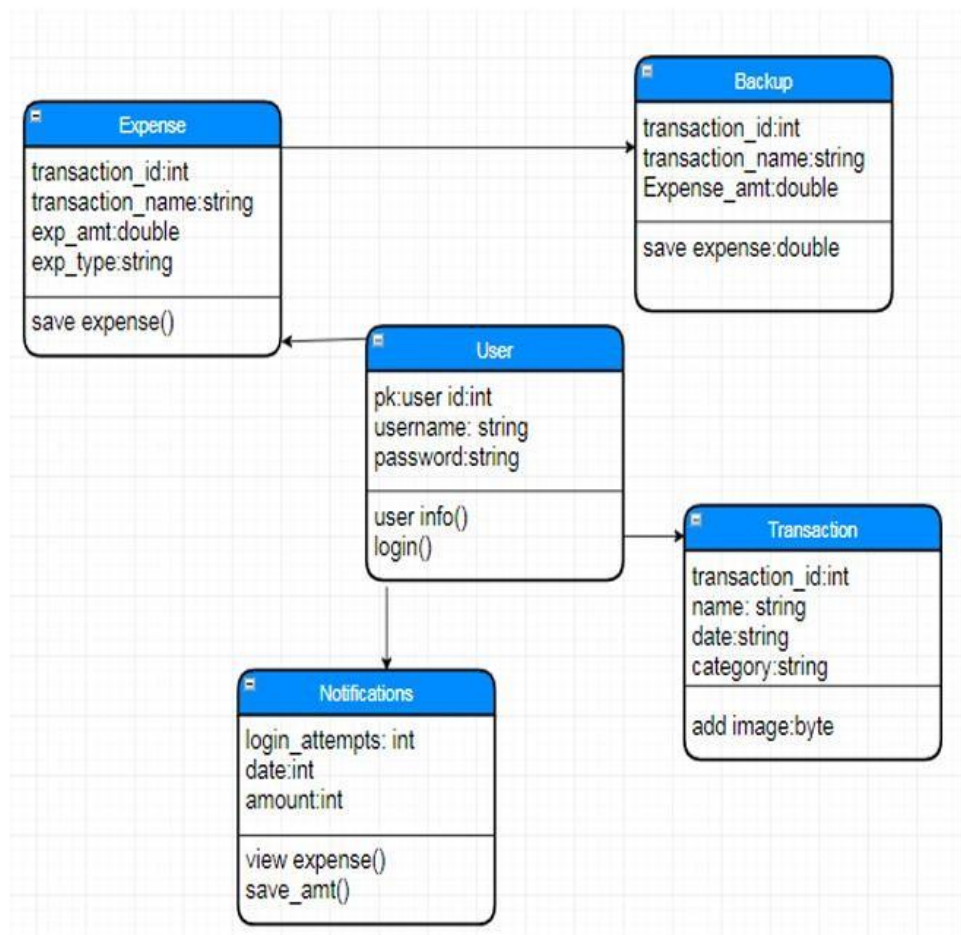


Figure 3.3.3.1 Class Diagram

It has five entities: expense, user, backup, notification, transaction, which have their own data members and methods. The above diagram shows the flow of functionality from entity to entity. Also, types of data members and methods of respective entity are mentioned above.

3.3.3.2 Sequence Diagram

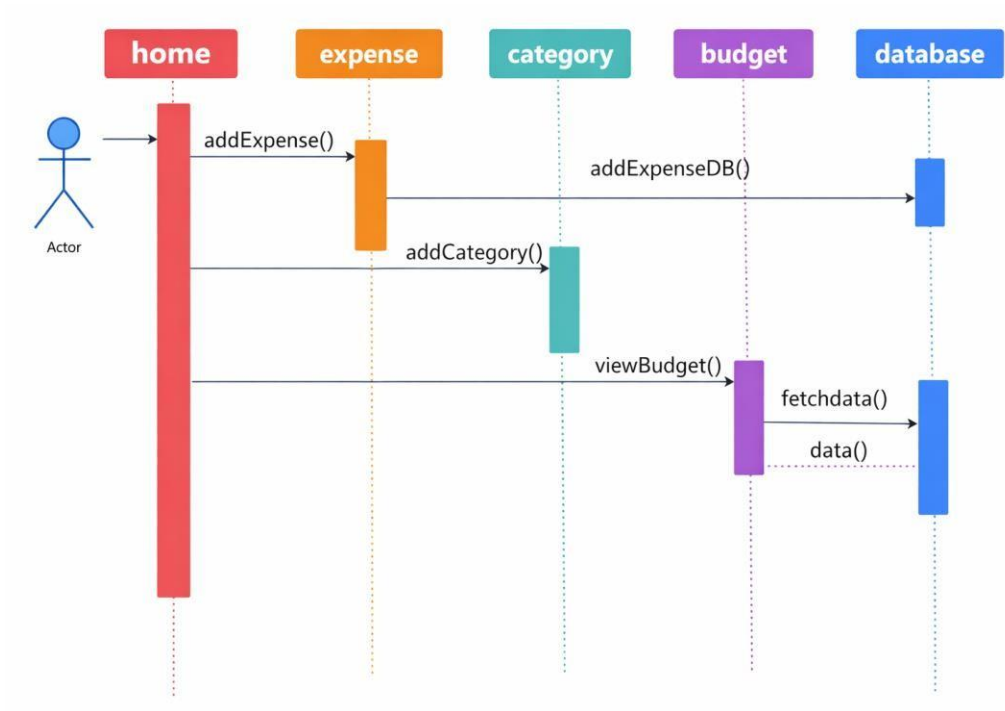


Figure 3.3.3.2 Sequence Diagram

3.3.3.3 Activity Diagram:

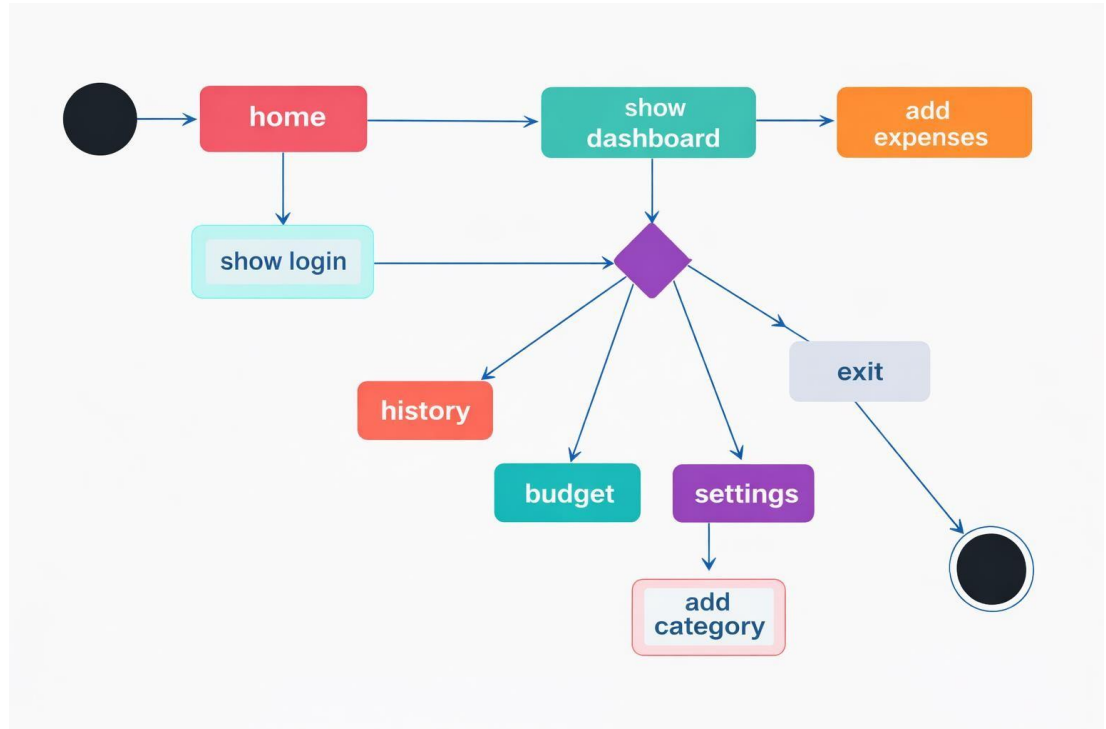


Figure 3.3.3.3 Activity Diagram:

3.3.3.4 Input output diagram:

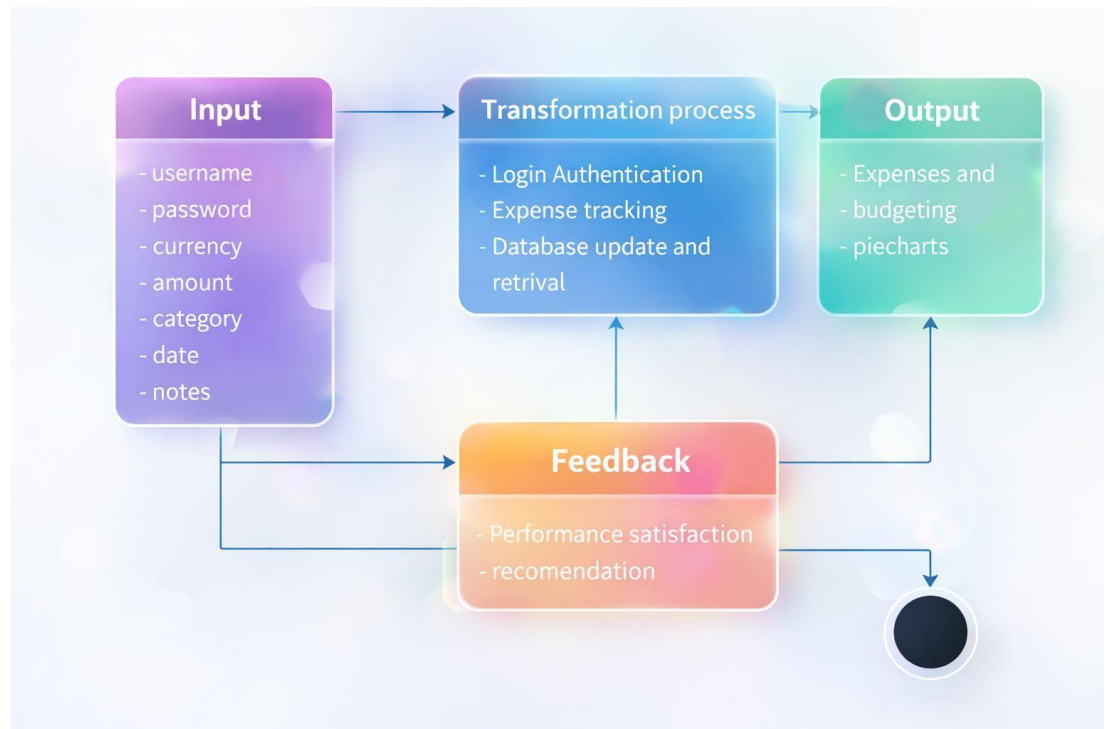


Figure 3.3.3.4 Input output diagram

The above diagram shows the major input that this system 'Pocket Tracker' in which the input includes username, password, currency, amount, category, date, notes while the transformation processing includes login authentication, expense tracking, database update and retrieval whereas output includes expense and budgeting and pie charts. The feedback includes performance satisfaction and recommendations.

3.4 Testing

Put simply, testing a piece of software means figuring out if it does what it should in all kinds of situations. You run the program or a part of it with particular data, and then see if what actually happens matches what you predicted. The main point of doing this is to find problems, or differences between what is supposed to happen and what does happen, in how the software works, how quickly it operates, how easy it is to use, and how safe it is.

Importantly, testing isn't just something you do at the very end of building the software. It should happen all the time during development. If you test while you're working on the initial requirements, the design and the actual code, you'll find and fix errors more easily, which will ultimately mean the project takes less time and costs less money.

3.4.1 Unit Testing

The Unit testing part of a testing methodology is the testing of individual software modules or components that make up an application or system.

S. No.	Test Case ID	Test Description	Input Test Data	Expected Result	Actual Result	Remarks
1.	TC-INS-01	Install PT app in android phone	Transfer PT app	Open application with its home page	Application executed with home page	Pass

S No.	Test Case ID	Test Description	Input Test Data	Expected Result	Actual Result	Remarks
-------	--------------	------------------	-----------------	-----------------	---------------	---------

1.	TC-LG- 01	Enter valid data in username and password field	ADARSH *****	Show home page for user Rasna	Displayed home page for user Rasna	Pass
2.	TC-LG- 02	Enter valid data in username and leave password field empty	ADARSH	Show Error	Didn't show any error	Fail
3.	TC-LG- 03	Leave username and password field empty and press login	*****	Show Error	Printed "Enter Username"	Pass
4.	TC-LG- 04	Enter invalid username and password	ADARSH *****	Show Error	Printed "You are not registered"	Pass

S No.	Test Case ID	Test Description	Input Test Data	Expected Result	Actual Result	Remarks
1.	TC-LG- 01	Enter expense values with their category	1500 with category clothing	Update category table with value 1000	Updated category table with value 100	Pass
2.	TC-LG- 02	Enter non-numeric value for expense field	ADARSH	Show Error	Printed "Enter Valid value"	Pass

3.	TC-LG- 03	Enter decimal value for expense field	155.65 with category food	Update category table with value 155.65	Updated category table with value 155.65	Pass
4.	TC-LG- 04	Enter negative value for expense field	-2635 with category rent	Update category table with value -2635	Updated category table with value -2635	Fail
5.	TC-LG- 05	Enter expense values without any category	1860	Update default category others with value 1860	Cannot update table	Fail
6.	TC-LG- 06	Enter future date for expense	2020/02/16	Show error in entering future expense	Updated table with future date	Fail

3.4.2 Integration Testing

- Integration testing is a really important step in checking a software program; at this point, you take the individual parts of the program (and they've each been checked by 'unit testing') and make sure they work correctly together. It's about how these connected parts communicate, confirming that information moves between them as it should, the links between them are correct, and the overall job of the combined sections does what you expect.
- The main goal of integration testing is to find and fix problems that happen when modules work with each other. These could be things like data being in the wrong format, methods being called incorrectly, data not getting through, or odd happenings because the modules' internal logic don't get along. Even if each part on its own is perfect, bringing them together can reveal errors in the way they 'talk' to each other.

- There are a number of ways to do integration testing. Top-Down Integration builds the system from the highest levels down, using ‘stubs’ as stand-ins for lower levels. Bottom-Up Integration starts with the lowest levels and slowly works its way up, with ‘drivers’ acting as higher level substitutes. Big Bang Integration throws almost everything in together and tests it all at once, which is quicker but can make finding what's wrong much harder. Incremental Integration adds and tests modules one at a time, so you can be sure each new addition doesn't ruin what was already working.

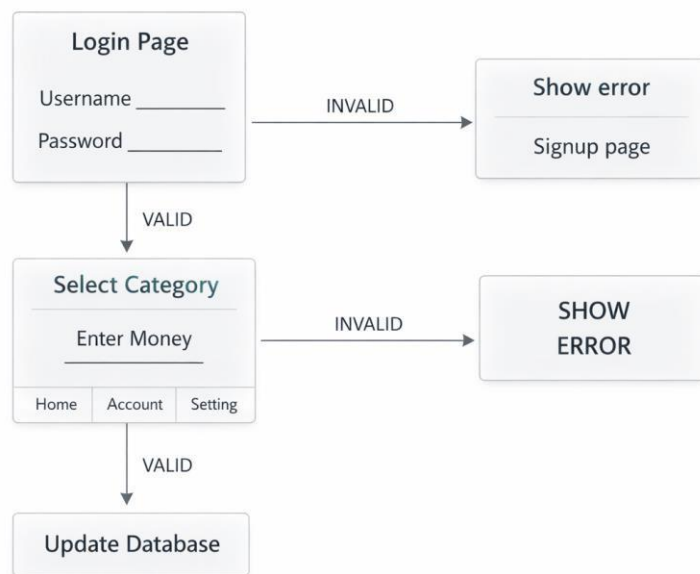


Figure 3.4.2 Integration Testing

3.4.3 System Testing

The system testing part of a testing methodology involves testing the entire system for errors and bugs. This test is carried out by interfacing the hardware and software components of the entire system and then testing it.

S. No.	Test Case Id	Test description	Input test data	Expected Result	Actual Result	Remarks
1.	TC-INS-01	Install PT app in android phone	Transfer PT app	Open application with its home page	Application executed with home page	Pass
2.	TC-LG- 01	Enter valid data in username and password field	ADARSH *****	Show home page for user Rasna	Displayed home page for user Rasna	Pass
3.	TC-DT- 05	Enter expense values without any category	1860	Update default category others with value 1860	Updated category others with value 1860	Pass
4.	TC-CL- 01	Go to chart page which shows the data in chart format	Click account tab	Update default category others with value 1860	Showed the chart of all the expenses of that day	Pass

3.5 Algorithms

- To log in, the system is designed to keep your password safe. When you type in your username and password, the system transforms (or 'hashes') what you typed, and then checks that hashed version against the hashed password already saved in our records. If they are the same, you get in. This way, your actual password isn't out in the open and your account is more secure.

- The process is: you enter your username and password, the system looks for that username, and if it finds it, it will compare your hashed password to the one on file. If they match, you're authenticated; otherwise, the login doesn't work.
- When you add a financial transaction to your account, the system needs to make sure everything is correct. It checks that all the details - amount, category, type, and so on - are sensible before it saves the transaction. And, it will update your overall totals.
- This is how it happens: you provide the amount, date, category and type of the transaction. Then the system verifies these are in the right format, cleans the information to prevent problems, adds the transaction to your history and finally, updates your income or expense amount in your profile.
- To help you stay on budget, the system watches how much you're spending in each of your categories. When you get close to the limits you've set, it will tell you.
- It does this by getting the budget amounts for each category, adding up all the spending in each category, and comparing the total to the budget. If you've spent more than 90% of the budget for a category, the system sends an alert, as an email or a notification in the app.
- Each month, the system can put all your financial information together into a report. This report shows you a breakdown of your income and outgoings, analyses things by category, and includes pictures of your spending.
- To make the monthly report, you first give the system your ID and the month you want. Then it gets all your transactions from the database for that month, calculates your total income and total expenses, and sorts the data by category to find totals for each. It gets the data ready for charts, like pie or bar graphs, and then makes a report you can download or export as a PDF or a CSV.

CHAPTER 4

RESULTS/OUTPUTS

4.1 User Registration and Login Output

This section of the program handles signing up and logging in, and does so in a way that makes sure only the right people get into the main parts of the system. Once someone has logged in correctly, they're automatically taken to their own dashboard, where they can get to their finances and all their information. The login process keeps things safe by scrambling passwords and carefully tracking each person's visit while they're on the site.

If a username and password don't work, the system gives a specific error message – for instance, “Incorrect Username or Password” or “Account Does Not Exist” - so you can figure out what's wrong and fix it quickly. The login box itself also checks what you type in to stop someone from using tricks or entering damaging code. And when you do log in successfully, a record of the login is kept; this is useful for checking things later and for security.

4.1.1: User Registration

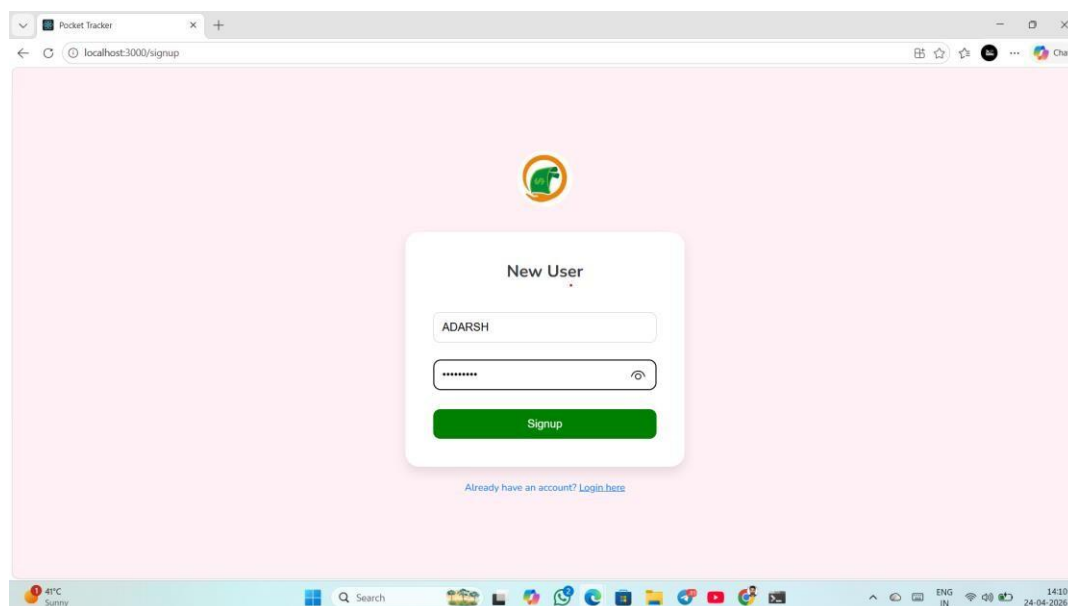


Figure 4.1.1: User Registration

4.1.2: User Login

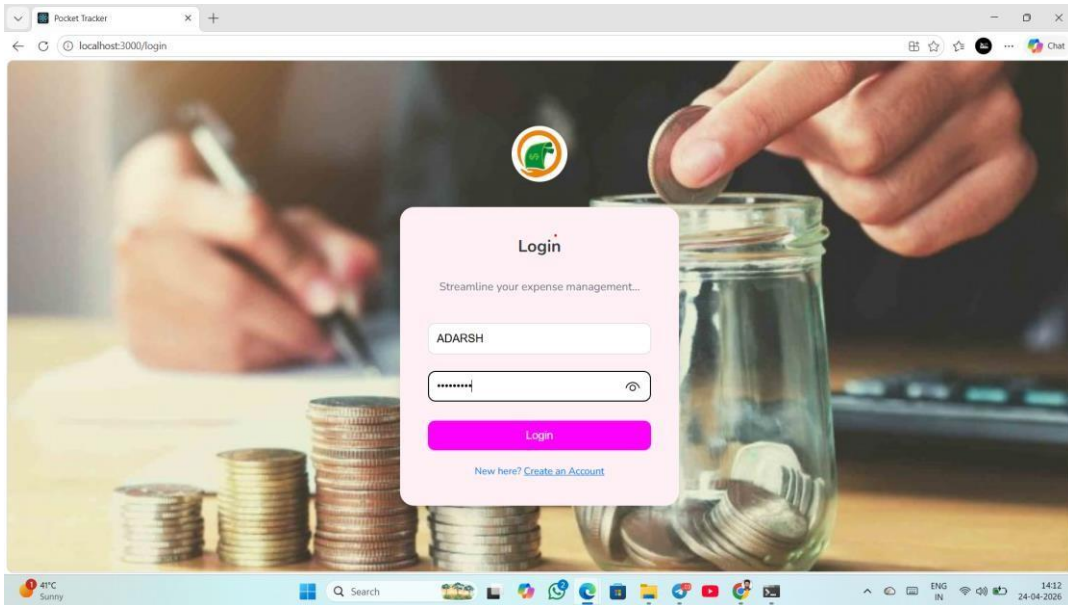


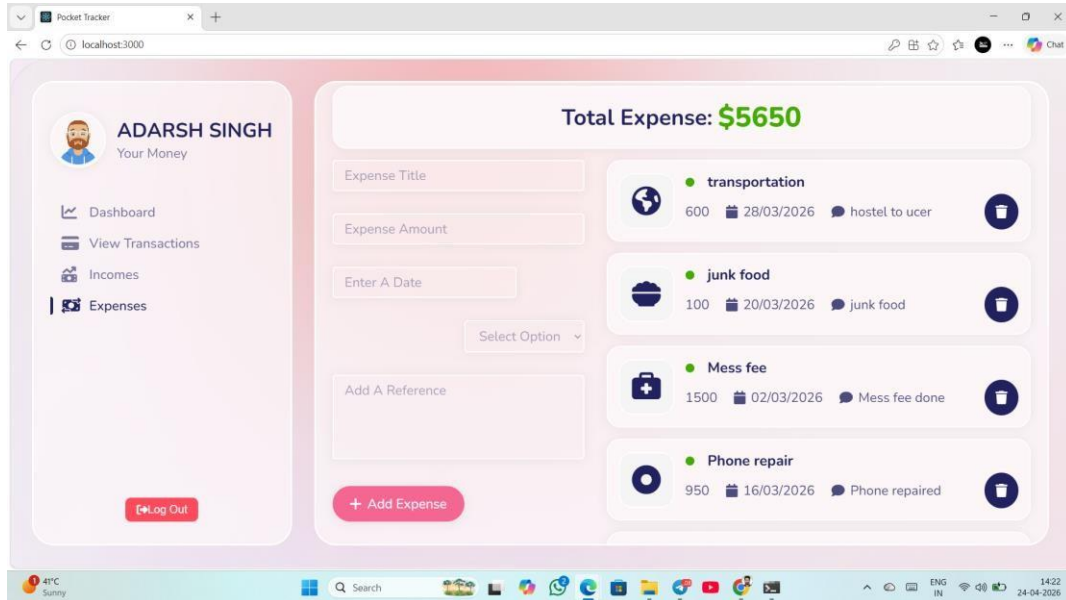
Figure 4.1.2: User Login

4.2 Add Transaction Output

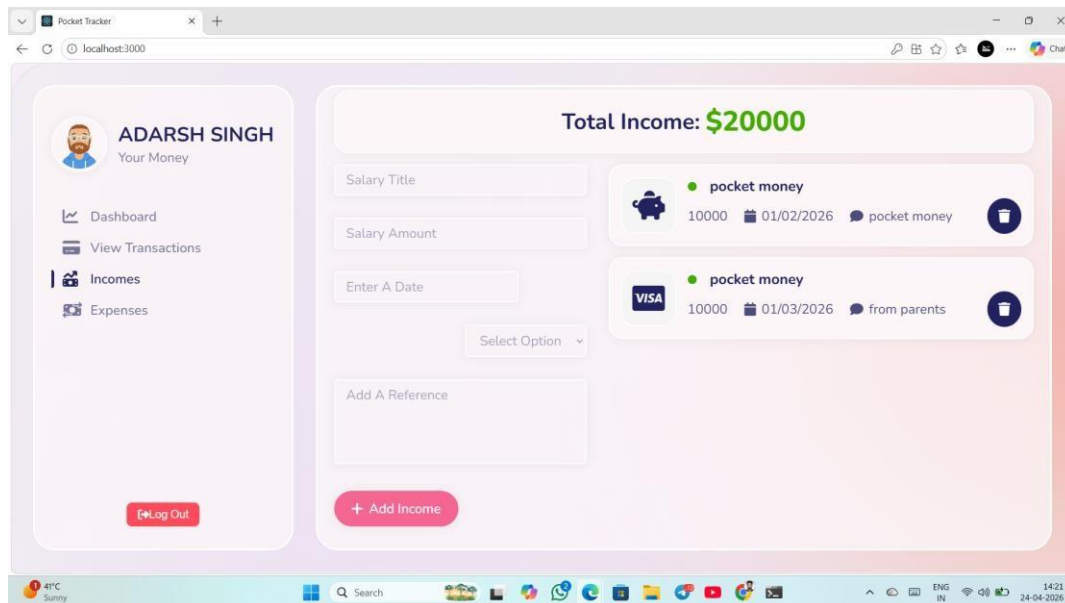
You use structured forms on the transaction entry screen to record your income and expenses. Each part of the form (category, amount, date, notes) has checks in it to make sure your data is correct before you send it in. When you do submit, the info is safely added to the database. This happens with Express.js and MongoDB running the work behind the scenes.

You'll see the transaction right away in your list of transactions; you don't have to refresh the page. React.js's way of keeping track of information makes this instant update possible and makes the whole experience feel quick and easy. Plus, things like colours and messages confirming your entries make it even more straightforward to use.

4.2.1: Add Transaction



4.2.2: View Transaction



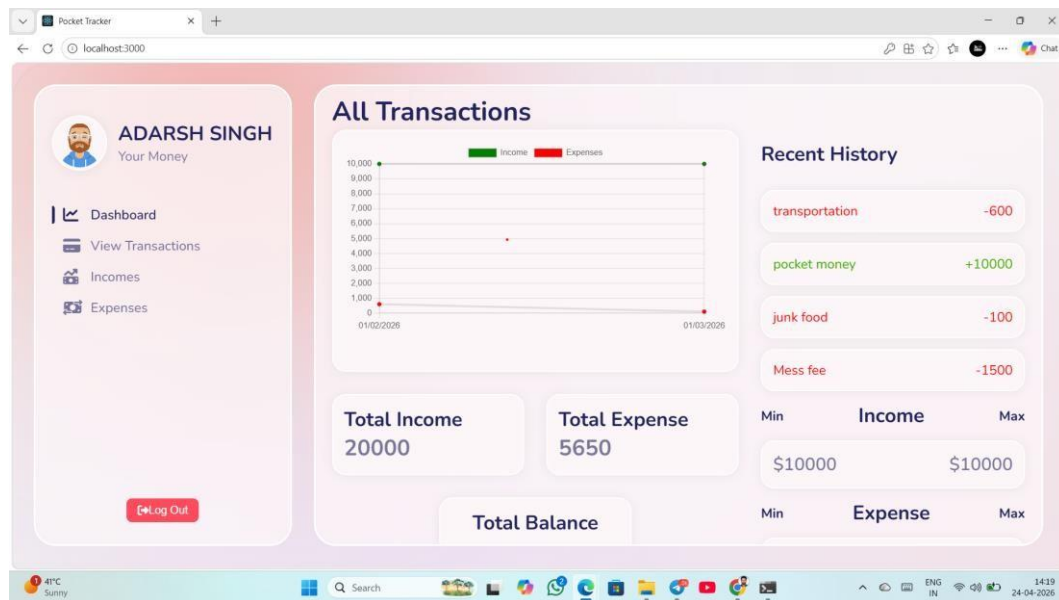
4.3 Dashboard Visualization Output

You can record your money coming in and going out through organized forms on the transaction entry interface. The dashboard then takes all of that transaction information and turns it into something you can actually understand. When you go to the dashboard, you'll see charts and graphs that change as your data changes, and specifically, you can see daily or weekly transactions as bars, and how your spending breaks down by category as a pie chart.

These charts update immediately when you add or change data, giving you a current view of where you stand financially. The charts are also colored (usually green for income, red for expenses) so you can easily tell the difference, and you get more details with tooltips when you hover over things, and interactive legends, but the screen doesn't get too busy. Each detail in the form - things like what something is for (category), how much it was, when it happened (date) and any extra comments (notes) - is checked to be correct before it's saved. Once it is saved, the data is safely put into the database by Express.js and MongoDB.

As soon as you add a transaction, it appears in your list of transactions on the screen, and you don't have to go to a new page to see it. React.js manages all the information to make this happen and for a quick, smooth experience. Little things like the colors and a message confirming you've saved something make it even easier to use.

4.3.1 Dashboard Visualisation



4.4 Budget Alerts Output

The system includes a proactive alert mechanism to help users stay within their financial limits. Each budget category is monitored in real time, and when expenditures exceed 90% of the predefined budget, a pop-up alert or push notification is triggered. These alerts notify the user of potential overspending, encouraging better financial discipline.

Furthermore, the dashboard features dynamic budget progress bars that visually depict how much of each category budget has been consumed. These progress bars change color (e.g., green to orange to red) based on the percentage consumed, making budget status instantly recognizable.

4.5 Report Generation Output

To facilitate financial planning and review, the system provides a robust report generation module. Users can generate monthly summaries of their income and expenditures, which are available in PDF and CSV formats for easy sharing and offline analysis.

Each report includes detailed category-wise breakdowns, including total spending per category, comparison with previous months, and graphical summaries. The reports are formatted for clarity and structured with headers, totals, and charts, providing both highlevel insights and granular details to support financial decision-making.

CHAPTER 5

CONCLUSIONS / RECOMMENDATIONS

5.1 Conclusions

Pocket Tracker, created with MongoDB, Express.js, React.js and Node.js (the MERN stack) is a big step forward in managing your money using current web tools. It's fundamentally meant to let people write down, look at in chart form, and understand where their money is going. The Expense Tracker has everything you'd want to properly oversee your finances in today's fast-moving world: you can log what you spend, group spending into categories, get up-to-the-minute spending summaries and even get smart advice for your money.

We've learned a lot during the building and launching of this project, and from both a technical side and what people using it actually want. These understandings have changed how the application is organized, what it does and how easy it is to use, so it really works for people in their daily lives and is built on a strong, adaptable software design.

5.1.1 Purpose and Relevance

- These days, with prices going up, the stock market going all over the place, and everything in general costing more, planning your money isn't just a good idea, you really need to do it. Actually, over 65% of people have a hard time sticking to a budget and more than 70% have no idea where their money goes each month, as several recent reports show. Expense Tracker is designed to solve this: it's a really easy to use system for getting a grip on your financial life.
- The app does lots of things. It helps you think about your spending, shows you how you're currently using your money, helps with budgeting and planning for the future, and gives you ways to save that are based on your own financial information.

5.1.2 Technical Proficiency and Implementation

Technically speaking, the MERN stack has been a really good choice for building an application that is both quick to respond to the user and able to handle a lot of growth. Each part of MERN did something specifically well. MongoDB's flexible way of organizing information was ideal for holding changing financial details and details about the people using the system, and because it's based on documents, it easily works with APIs using JSON. Express.js was the framework for the backend - it handled directing things efficiently, letting us add extra tools (middleware), and building APIs in a RESTful way. React.js made the front end of the application feel alive and respond to the user. With features like hooks, Redux for managing the application's status, and showing only what should be shown at any given time, the screen layout is easy to use. Node.js ran the code on the server, and its ability to do multiple things at once without waiting for each one to finish made the application faster and easier to expand. Beyond the main MERN tools, we also used Chart.js for graphs and reports, bcrypt to securely store passwords, JWT to verify users' identities, and Tailwind CSS for the look and feel of the interface. These were all added to create a strong and secure platform.

5.1.3 User-Centric Design

We really focused on making people who use the project happy, using the best ideas for how things should look and work (UI/UX). The app is for everyone, from people who are comfortable with technology to those who aren't. It includes a dashboard that's simple to get around, charts you can interact with and expense types shown in colours, a design that works on phones and tablets, the ability to change the look (to something brighter or darker) and helpful hints and reminders. All these things together are meant to make the experience of using the app both nice and helpful. During the test runs, people said how important it was for things to be simple and for them to be able to change the app to how they like it, and so we made sure to do that in the later versions of the design.

5.1.4 Functional Strengths

What's really good about the Expense Tracker is how much it does. You can put in your expenses, but on top of that it sorts your spending into different types like groceries, transportation, entertainment, and so on. It shows you how your spending changes each month or week, warns you if you're going over budgets you've already decided on, and even gives you ideas to save money based on what you've spent before. It also handles several accounts and backs up all your expense information. Because of all this, it's much more than just a way to write down what you've spent, it's like having a smart helper for your money.

5.1.5 Scalability and Performance

The project has done a really good job of working smoothly, and is designed to handle a lot more as it goes on. Its inner workings will let you add more and more resources simply by adding more machines, and it uses Docker for all that. MongoDB can divide its data across multiple places (a trick called sharding) and the website itself has been made to load as quickly as possible.

With just a little bit of tweaking right now it could manage loads of people using it, connect to banks' systems, and watch your finances as they change using either sockets or webhooks. Because of all of this, the Expense Tracker isn't just good for you to use on your own, but can also become a service you pay for and lots of people use.

5.1.6 Challenges and Resolutions

Every project, naturally, has its hurdles. During the building of this one, we ran into a few things. Getting information from one part of the application to another and keeping track of what's going on with the whole thing were a bit tricky at first, but we solved them with Redux. Security was important, and we handled that with JWTs for people to log in and a process of scrambling passwords. We used Mongoose to check data as it's being saved, keeping everything in the database as it should be. And to make things go faster, we made the files smaller, only loaded

parts of the application when needed, and squeezed the data as it travels back and forth. Getting past these difficulties really strengthened the application and made it work far more predictably.

5.1.7 Future Scope

Pocket Tracker's off to a great start, but there's loads more it could do. We could use the cleverness of machine learning to look ahead and guess what you'll spend, and then give you savings advice just for you. Imagine being able to simply tell Alexa or Google Assistant what you've spent. We're also thinking of making saving more fun with things like saving challenges, rewarding you for how long you manage to save (financial streaks!) and earning badges. And to reach many more people? It should be available in many languages. Plus, a phone app, built with React Native to work on all sorts of phones, is on the list.

5.2 Recommendations

5.2.1 Introduction

Pocket Tracker is a complete web app, built with MongoDB, Express.js, React.js and Node.js. It's a modern application for the web that lets you easily add your spending, keep an eye on it, and sort it into types. What's more, you get in-depth reports and clever ideas for handling your money in a better way. Because knowing about money and having a way to budget is becoming more and more important, Pocket Tracker could be really useful in everyday life.

This part of the document will give a lot of specific advice to improve Pocket Tracker's features, how quickly it works, how nice it is to use, and how much it can grow. This advice comes from what's normal in the business, what's new in apps for managing your own money, and what people expect from digital tools these days.

5.2.2 Functional Recommendations

5.2.2.1 Integration with Banking APIs Rationale:

Lots of budgeting apps make you do something that's a bit of a pain: you have to get all your spending details from your bank and type them in. But if the app connected to services like Plaid, Yodlee or Salt Edge (companies that gather financial information), all of this would happen for you, your transactions would magically appear, and you'd have much less to do!

What would be the result of this? Well, it would do a lot more for you without you lifting a finger, it would be much less likely you'd accidentally get your numbers wrong when typing, and you'd probably use the app a whole lot more. In fact, being able to do this would make the app feel much more trustworthy and actually good at helping you.

However, it's unbelievably important to do things by the book with all the laws and regulations, and to absolutely get the user's permission.

5.2.2.2 Support for Multiple Currencies and Exchange Rates

Because we live in a world where everything is interconnected, people commonly use lots of different currencies. For people on the move, whether they're on vacation, working as freelancers, or with the laptop-as-home lifestyle of digital nomads, a way to keep track of spending needs to understand this.

As for how to do this, you could tap into services like OpenExchangeRates or Fixer.io to get current exchange rates. Users should absolutely be able to choose their 'home' currency. And then, when they're listing how much things cost, the app should keep that expense as both the amount in the currency used at the time, plus the equivalent in their chosen home currency.

What will this achieve? Well, it'll bring in a much wider range of people from all over the world, and be far more useful to anyone who spends money in different countries.

5.2.2.3 Recurring Expense Recognition and Automation

We all have things we pay for over and over – things like subscriptions, rent, and all those monthly bills. If the app could automatically spot these and suggest they're going to happen again, it'd be so much nicer to use.

To do this, the app will look for patterns in your spending (either by learning from it, or by following set rules).

And you'll be able to say 'yes, that's right' or tweak the details of these regular payments.

The good part of all this? You won't have to type the same information in all the time, and you'll get more correct records of your spending.

5.2.2.4 Smart Notifications and Budget Alerts

Getting a little “nudge” from alerts and reminders can really help you get a handle on your money. Think about being told when you've gone over your spending limit for, say, groceries or clothes, noticing if your card is being used in a way that's not normal for you, or a heads-up before things like your phone bill are due.

To make these happen? We'd use Firebase Cloud Messaging (FCM) to send notifications to your phone, and Node.js “cron jobs” to send those reminders on a set schedule. Ultimately, this means you'll start to plan your budget in advance, and generally develop a healthier way of dealing with your finances.

5.2.2.5 Advanced Filters and Dynamic Search Features

The more information we have about what people do, the more vital it is to be able to locate and sort through it all.

For filtering, we should allow people to narrow things down by dates, what kind of item it is, how much it cost, who they paid, and how they paid. What's more,

they'll want to use a mix of these - like being able to find all grocery purchases from March that were over one hundred dollars.

We'd build this with MongoDB's aggregation framework and the searching elements in React.

As a result, the application will be much easier to use, people will like it a lot more, and it'll look and feel far more polished and professional.

5.2.3 UI/UX Recommendations

Most people now use phones and tablets to manage their money. Because of this, we're building the look of the application for small screens first, and then making it work beautifully as it gets bigger on tablets and computers.

How we're doing it: We're using tools like Tailwind CSS or Bootstrap which change how things are laid out on the screen, using 'media queries' to do this, and adjusting elements on the fly. We'll also do lots of testing - on the screen size selectors in browsers, and importantly, on actual phones and tablets themselves - so it all works exactly as it should, on everything. And we'll make sure it's easy to tap and swipe with your thumb, and that the screen isn't overly full.

What's the point? A design that responds to the size of the screen makes the application much easier for everyone to use, and it's easy to use on any device. This will encourage people to use it more and to keep coming back.

5.2.3.2 Dark Mode and Custom Themes

Many people find staring at a screen much easier on the eyes with dark mode, particularly if they are using it for a long time or in a dim room, and these days, users pretty much want it. Being able to use a theme you've designed yourself will make people happier with the application.

To actually do this, we'll use CSS variables so changing themes and switching between dark and light is simple.

We can remember each person's theme choice in their browser's storage (localStorage) so it's there when they come back, or we could have the server handle it and pull the theme from MongoDB.

Users should be able to choose their theme themselves, or have the app automatically go with whatever theme the phone or computer is using.

Adding dark mode and customisable themes will make using the application a much better experience, help more people use it (accessibility), and show that the application is thoughtfully and professionally designed.

5.2.3.3 Visual Representations: Color-Coded Categories and Charts

We understand things presented to us as pictures, graphs and coloured markers a lot quicker than if they're written out in a report. And when a picture is designed thoughtfully, it helps us choose the best course of action.

To make those pictures, you could use Recharts, Chart.js or D3.js; these are all programs for building charts that respond to what you do, with pie charts, bar graphs and lines showing trends being a few possibilities.

Importantly, use the same colours for the same things each time - for instance, green to show money coming in, and red for money going out.

Because of using graphics, what's happening with finances is much easier to understand, it helps people manage their money, and it makes the application nicer to look at and simple to navigate.

5.2.3.4 Accessibility Enhancements

Designing for everyone, meaning people of all abilities, is the core of creating an application truly anyone can use. And meeting accessibility guidelines like WCAG doesn't just open the app to a lot more people, it's also the right thing to do.

To actually do this? Well, add ARIA labels so screen readers can interpret everything properly, allow people to get around the application using only the keyboard (some

folks can't use a mouse, you see), and maybe for a later version, we could even include voice commands to make things even easier.

Because of these accessibility updates, many more people will be able to use the app, it will be a more welcoming place for all, and it will help us with the legal and moral expectations of how software should be.

5.2.4 Security Recommendations

5.2.4.1 End-to-End Data Encryption

Because we're dealing with people's private money details, keeping that info safe from people who shouldn't get it and possible data leaks is absolutely essential.

To achieve this, use HTTPS with SSL certificates so the data is safe as it travels across the internet. For passwords, use bcrypt to scramble them, so even if someone did manage to break into the database, the passwords wouldn't be readable. And within MongoDB, lock up important bits of information – things like what's in a transaction's notes, and someone's personal details - with encryption.

Good strong encryption is what gives people confidence in us and is the basis for an application that is really secure.

5.2.4.2 Social Logins

Letting people sign in with their existing social accounts makes getting started much easier, and it's actually more secure because it relies on the security of well-known companies like Google, Facebook, or GitHub.

To do this, we should use Passport.js to add the ability to log in with Google, Facebook, GitHub. And, for anyone who wants to, we'll still have the usual email and password login.

Because of these social login choices, people will find it quicker and simpler to join up, get going, and they'll get the benefit of better security – things like two-factor authentication and a record of their logins being watched for anything unusual.

5.2.4.3 Role-Based Access Control (RBAC)

As the app grows, we're going to need to control who can do what – and RBAC (Role Based Access Control) is how we'll do that. It makes sure people can only get at the info or do the things they're supposed to.

Let's look at some examples. An admin gets absolutely everything: they can manage users, fiddle with all the settings and look at the analytics. A regular user? They only see their own dashboards and reports. Moderators can see what users are up to and manage the community aspects. Support Agents will be able to answer questions and solve technical issues.

To actually make this happen, we'll assign roles within MongoDB and then use Express.js 'middleware' (think of it as a gatekeeper on each webpage 'route') to make sure the right people have the right permission.

And what's the result of all this?

5.2.5 User Engagement Recommendations

5.2.5.1 Gamification Elements

Give people badges when they put in their spending each day or hit their savings goals. To get people to compete in a nice way, show leaderboards. And for the really brilliant at saving or tracking expenses, how about names like "Hero Saver" or "Pocket Master"?

Basically, turning things into a game makes the app more enjoyable, gets people to keep using it, and means they'll open it regularly.

5.2.5.2 Community Features

Let people chat with each other on forums, boards for swapping money-saving ideas, or in a questions and answers section. They should be able to share what they've found to save money or how to manage their finances.

The good thing about having these ways for people to get together? Well, people learn from one another, feel like they're a part of something, and will use your site a lot more.

5.2.5.3 Feedback and Support Integration

You'll be able to get help with any problems, instantly, through a chat right inside the app. And to continually make things better, we'll ask you for your opinions on what's working, and what's not, with the features and how easy the app is to use, with fairly frequent surveys. For really great support and to keep track of customers, we'll be using services such as Intercom, Crisp, or Zendesk...all of them are established and will link nicely with our customer management system.

CHAPTER 6

APPENDICES

The appendices have extra details to back up how the Pocket Tracker project was created, checked, and put into use. Things like architectural diagrams, the design details, pictures of what you see on the screen, explanations of how the testing worked, and other important stuff that didn't fit in the main parts of the report are all included. Essentially, each appendix gives you a more thorough grasp of the project's construction and operation.

Appendix 1: System Architecture Overview

Pocket Tracker is built using the MERN stack, a collection of technologies that work together. The part you'll actually see and interact with, the front end, is made in React. It's designed to look good on all devices and is visually done up with either Material UI or Tailwind CSS. This section is responsible for what you do, checking that your entries make sense, and showing your information.

The behind-the-scenes work is done by a backend using Node.js and Express.js: this provides all the 'addresses' for the app to respond to your requests (authentication, managing your spending, and keeping tabs on your budget). Between your requests and the database, the backend has a safety check and deals with anything going wrong.

User accounts, what you've spent, how you've grouped your spending (categories), and your budget limits are all held in MongoDB Atlas. Mongoose makes sure all the information in the database is organized and correct.

Appendix 2: Database Design and Collections

Think of MongoDB as filing things into folders called collections. We have one for Users with each person's ID, their name, email, a scrambled version of their password, and the date they joined. Then there's Expenses, and inside each expense we store its ID, who spent it (the user ID), how much it was, what it was for (the category), a bit of explanation and the date. Categories are a folder for different types of spending, with a category ID, the user who created it, and the category's name. Budgets is a collection with each budget's ID, the user it belongs to, the month, the maximum you can spend, and how much you have spent.

Finally, Notifications tells you things, and for each notification, we have its ID, for which user, what type of alert it is, the actual message and when it happened.

Appendix 3: API Design and Endpoint summary

To create a new account, you'd send something to `/API/auth/register`, and to actually get in, you send details to `/API/auth/login` and you will get a JWT token.

When it comes to your spending, you can add something to `/API/expenses`, and see everything you've spent at `/API/expenses`. You can amend a particular expense by sending changes to `/API/expenses/:id`, or get rid of one entirely by deleting at `/api/expenses/:id`.

For budgeting, you establish your spending limit for the month with a request to `/api/budgets`, and get the details of a specific month's budget from `/API/budgets/:month`.

And finally, for a look at where your money goes, `/API/analytics/summary` shows you how much you've spent broken down by category, while `/API/analytics/trends` displays your spending over months.

Appendix 4: Interface Screens and User Journey

To get in, you'll use a login or sign-up page that's safely protected with JWT authentication. Once you're in, the dashboard is where you'll get a general picture of how much you're spending, what your budgets are, and everything shown in charts. You can add new transactions, change existing ones, or get rid of any in the expense entry form. And if the default options don't work for you, category management allows you to make your own spending categories. For planning ahead, the budget planner is for deciding on monthly limits and seeing how you're doing. The analytics page shows your spending habits using bar and pie charts, and the notifications panel will give you immediate warnings when you're going over budget or if something has been updated.

Appendix 5: Testing Process and Validation Summary

We checked the backend functions (like creating, reading, updating and deleting expense information) using Jest or Mocha. Postman was the tool for testing our API endpoints. And for the user interface, we used React Testing Library to make sure forms did what they should with their validation and that everything was displaying properly.

On the front end, React Hook Form or Formik do the job of looking at what you've typed in. The backend employs Joi to ensure all the data matches the expected format.

Plus, we've got specific error messages for you if you enter something wrong, your login details have run out of time, or the API itself hits a problem.

Appendix 6: Deployment and Environment Configurations

To get things live, the part of the application users see (the frontend) will be on Vercel or Netlify, and every time you make a code change it will automatically be updated - that's CI/CD. As for the engine of the application (the backend), it will be hosted on Render or Heroku, and all sensitive information (your secrets) will be safely tucked away in ``.env`` files. MongoDB Atlas will be the database, and it'll be able to get bigger as you need it.

To make everything behave in the right way, we're using a few settings. ``.NODE_ENV`` will tell the application whether it's running for development or for when it's actually used by people.

Appendix 7: User Feedback, Limitations, and Future Scope

People really like how simple the interface is, the fact that it works well on any device, and how helpful the breakdowns of your spending are.

As for improvements, people would love to have a bit more control over what categories they use, to be able to use it when you're not connected to the internet, and to link it to their bank accounts.

Right now you can't connect to banks or digital wallets at all, and for pretty much everything to work, you need internet. Also, getting help or finding in-depth explanations isn't particularly easy.

Looking ahead, the app could use artificial intelligence to guess at future expenses and offer suggestions. It should handle lots of different currencies and be adapted for use in different regions, and ideally, the app would work brilliantly even without the internet, storing everything on your device and then updating when you're back online.

Appendix 8: Plagiarism Report

The results from checking for plagiarism in the Pocket Tracker project report are here. Basically, we wanted to prove the work is the student's own and that they've followed all the rules about being honest in their studies.

What you'll find inside is a detailed "similarity index" - that's a number from the plagiarism-checking program showing how alike the work is to other things. Sections that the program found to be matching something else are pointed out, and explanations for any proper similarities are included: technical language, where sources are quoted, or in the bibliography.

The good news is the report shows the project is a true and original piece of work, with hardly any matching to anything else out there, so it's totally in line with academic honesty.

Appendix 9: Project Diary

From June 2025 to May 2026, this section is a sort of diary of how the project went each day.

The way it's put together is by date, and for each period, it looks at what we achieved as big steps, the difficulties we ran into, and how we solved them.

In June and July of 2025 we were finding out exactly what was needed, seeing if it was possible to do what was needed, and getting the very first foundations of the MERN system in place. Then, from August to October we designed how the database would work, built the way for different parts of the application to talk to each other (the API), and built the login section. November 2025 through January 2026 were for the part of the program users actually see, created with React.js - we improved how it looks and feels, and got it working with the 'behind the scenes' services. February and March 26 saw all the testing (of individual bits, how they work together, and the whole system), repairing errors, and making it run as quickly as possible.

Appendix 10: References to External Resources

The numerous books, research, and websites were looked at while designing, creating, and writing about the Pocket Tracker application. They gave the project its foundations in theory, technical know-how, and useful, real-world ideas, all beyond what you'll find specifically mentioned in the main report.

The list of references is important because it shows the careful, thorough research that went into your project and gives it academic weight.

REFERENCES

- [1] P. Thanapal, M. Y. Patel, T. P. L. Raj, and J. S. Kumar, “Income and Expense Tracker,” *Indian Journal of Science and Technology*, vol. 8, no. S2, Jan. 2014. ISSN: 0974-5645.
- [2] G. Bekaroo and S. Sunhaloo, “Intelligent Online Budget Tracker,” in *Proceedings of the Computer Science and IT Education Conference*, 2014.
- [3] Y. Anitha, R. Ranjini, and S. Gomathi, “Easy App for Expenses Manager,” *International Journal of Computer Techniques*, vol. 3, no. 2, Apr. 2016. ISSN: 2394-2231.
- [4] S. Chandini, T. Poojitha, D. Ranjith, V. J. M. Akram, M. S. Vani, and V. Rajyalakshmi, “Online Income and Expense Tracker,” *International Research Journal of Engineering and Technology (IRJET)*, vol. 6, no. 3, March 2019. e-ISSN: 2395-0056, p-ISSN: 2395-0072.
- [5] A. Dixit and M. Patel, “Personal Finance Management Applications: A Review on Features and Security,” *International Journal of Computer Applications*, vol. 182, no. 21, Oct. 2018. DOI: 10.5120/ijca2018917952.
- [6] J. B. Rocha and R. Silva, “A Comparative Study of Mobile Applications for Personal Finance Management,” in *Proc. 18th IEEE Int. Conf. Mobile Data Management (MDM)*, Daejeon, South Korea, 2017, pp. 312–315. DOI: 10.1109/MDM.2017.57.
- [7] A. Sharma, P. Kumar, and S. Gupta, “Expense Management Mobile Application Using React Native and Firebase,” *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, vol. 6, no. 2, pp. 112–117, Mar.–Apr. 2020. ISSN: 2456-3307.
- [8] K. Vyas and N. Shah, “A Secure Mobile Application for Managing Personal Finance,” *International Journal of Innovative Research in Computer and Communication Engineering*, vol. 5, no. 6, pp. 12345–12350, Jun. 2017.

Shyam Bahadur Verma

PacketTracer.pdf

 International Conferences @ UCER

 UCER-2026 Research Publications

 UNITED GROUP, PRAYAGRAJ

Document Details

Submission ID

trn:oid::1:3552151108

Submission Date

Apr 27, 2026, 4:40 PM GMT+5:30

Download Date

Apr 27, 2026, 4:53 PM GMT+5:30

File Name

PacketTracer.pdf

File Size

1.2 MB

64 Pages

14,721 Words

71,052 Characters

61% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

-  **103 AI-generated only 61%**
Likely AI-generated text from a large-language model.
-  **0 AI-generated text that was AI-paraphrased 0%**
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



The Ghosts in the Dataset: Hallucinations in A.I. as Gothic Narrative

Adarsh Singh
(2200100100025)

Department of Computer Science and Engineering
United College of Engineering and Research,
Prayagraj, India
singh9118adarsh@gmail.com

Abstract:

LLMs, or Large Language Models, are amazingly good at sounding like people. However, they're prone to 'hallucinations' - boldly stating things that are completely made up, and this is a problem for both how safe AI is and how trustworthy its stories are. This paper looks at hallucinations through the lens of Gothic stories and posthumanist ideas. We're thinking of them as bursts from a "digital unconscious", with the huge amounts of data they are trained on being like spooky old manor houses. The 'latent spaces' inside the model, where the core of its understanding is, are like confusing, broken-down labyrinths that hold onto hidden, spectral memories. By really digging into specific instances where they go wrong, we've found these mistakes are like new, unsettling Gothic monsters, and that a machine being unreliable actually shows us our own human anxieties, past hurts, and how truth is becoming scattered in this digital world.

Keywords:

Large Language Models (LLMs), Transformer architecture, Common Crawl dataset, Data noise and contradictions, Opal Ritual, Posthumanist theory, Labyrinthine ruins.

I. INTRODUCTION

Large Language Models (LLMs) are amazing at writing in a way that sounds like something a person would say, but they're prone to "hallucinations" - confidently stating things that are

simply not true - and that really throws into question how much you can trust them for things like writing or research. What's being done right now to fix this is to look at the technical side of things, with the aim of getting better training data or a more sophisticated way of getting information, as if these hallucinations are just bugs to be erased. But, this misses something much more interesting: these hallucinations in LLMs actually show a deeper way they operate, and it's a way that's surprisingly like the plots of Gothic novels.

Think about Gothic novels, from the earliest one, The Castle of Otranto, right through to Dracula - they all thrive on narrators you can't be sure are telling the truth, on "discovered" papers, and on buildings that feel haunted, building tension from bits and pieces of truth and a general feeling of unease. And LLMs do the same sort of thing: their internal "thinking space" is like a confusing, sprawling Gothic mansion where how words are linked leads to 'hidden routes' of completely made up details, and all the information they've been trained on is a kind of digital burial ground of conflicting, random things on the internet that pop up again when you least expect them.

This paper looks at the technical side of hallucinations in AI created fiction, but at the same time points out how they fit with the techniques used in Gothic storytelling. We're calling a hallucination anything the transformer model comes up with that sounds right but isn't is the result of gaps in the information it has, biases in the data, or because of the way it works with extremely complicated sets of numbers (embeddings) and probabilities. The main ways it happens are that it predicts the next word in a sentence to make it read nicely,

and doesn't worry as much about whether it's true, and because it's been trained on the whole of the internet (like Common Crawl) and that's full of stuff that's old, contradictory, or just incomplete.

We have three aims in this study: to find out how the way the model is built leads to these invented stories, to find where the mistakes come from in the data it was taught with, and to look at what effect they have on the fiction it generates.

Problem Statement:

LLMs, these huge language programs, will confidently make things up in their stories — entirely false details about what happened, who people are, or even what something refers to — and they weave these lies right into the actual facts. Trying to fix this with technical means, by scrubbing the data they learn from or changing the models themselves, just sees these fabrications as bugs to squash.

But the real issue is how transformers (the way LLMs are built) operate. They predict the next word, one at a lovely, flowing sentence at a time, and being smooth and sensible is more important to them than being correct. Because the huge collections of text on the internet that they're taught from are full of holes and self-contradictions, the models will naturally invent details to fill in the blanks and keep the story going. This is much like a Gothic novel.

Think of something like Horace Walpole's *The Castle of Otranto* and characters finding bits of old paperwork in falling-down houses, which then lead them to very dodgy understandings of what happened previously. LLMs do the same thing: their internal 'thinking' space has connections which are like secret rooms holding broken bits of data, and the internet text they're trained on is a massive, confused, digital archive. So the model becomes a storyteller you can't trust, stating its made-up details as if they're things it's found to be true.

This push and pull between a sensible sounding text and being accurate is still present in all LLMs.

They're mainly taught to make sentences that read well. And until we deal with the fact that the whole architecture of these models predictably creates these Gothic-style narratives, anything we do to lessen the problem is only treating symptoms. We won't get to the bottom of the problem with how they create language by chance.

Scope and Relevance of the Study:

This work is a look into the weird things AI does when it makes up stories, specifically how 'transformer' models will invent things when they don't have all the information. He's looking at what's going on in the model's 'thinking' space, the problems with the jumble of information they're trained on from the entire web, and doing specific tests around areas where the AI is likely to be lacking in details about the past. And interestingly, the way these AIs work is a bit like old Gothic novels, with their stories built from dubious and broken pieces of information from dusty old places - though he isn't thinking about how to fix this, how it works for factual stuff or making the models better.

These 'hallucinations' are a core problem with Large Language Models as they are now; their need to sound good actually makes use of flaws in the information they've been fed. Common Crawl and similar collections of stuff from the internet, are huge on quantity but not on quality. They're full of old, gone websites, forums that are no longer there, and information that says different things, and the AI will use all of this to sound sensible, not to be correct. And, conveniently for the companies making them, a massive, messy collection of data lets them create very large models that are impressively articulate...and get more money and power, even if what they say isn't actually true.

This problem is easiest to see in AI's attempts at creating fiction. If you ask it about something very little known from history, the AI will confidently come up with details that seem right - names, dates, what people did - based on the statistical likelihood of those things appearing in the information it has, rather like characters in Gothic novels building complicated stories from incomplete and crumbling old documents. This is useful for the companies, as it shows off how well the AI can use language, and leaves the problem of the incorrect information for the people using it.

The link to Gothic novels points to the moral side of this: just as those unreliable narrators in Gothic fiction confidently tell you things that aren't true, the AI uses the faults in its data to make itself sound good.

II. LITERATURE SURVEY

Hallucinations, as far as the tech world is concerned, are when Large Language Models boldly state incorrect 'facts'. This happens because of the data they were taught on and the way they are built (with something called a transformer). It turns out the transformer's method of predicting the next word in a sentence is more about what's likely to be there, than what is actually true. So, if a model doesn't know much about something, it will just make it up. And a lot of the data they are taught on, from the huge Common Crawl collection, is pretty messy, at least 60-70% of it. Think of all the posts on forums, blogs, and pages that have been taken down. This mess creates links in the model's 'thinking' which then allow it to make mistakes with great confidence. Trying to fix this with retrieval-augmented generation (RAG) or further training does make the model sound a little more sensible, but it doesn't solve the underlying issue.

In Gothic novels, authors use a story that's all in pieces to make things scarier. A straightforward example: a researcher finds old letters in a falling-down country house, detailing a family being torn apart. But the letters don't agree - one speaks of a fire in 1780, yet another alleges poisoning in 1782. The person telling the story then pieces it all together into a dramatic version of murder and ghosts, and presents their guesses as though they are the truth, and conveniently overlooks what doesn't quite fit. The house itself, with its gloomy hallways and locked rooms, is like those incomplete records, guiding the reader down paths that aren't to be trusted and toward something truly horrifying.

The way transformers internally 'think' (their latent spaces) are surprisingly like those Gothic houses. The way words are represented as numbers (token embeddings) make 'secret passageways' of

statistical connections. Because of this, actual facts (like towns where they dig for opals) become linked with things that are completely invented (blood sacrifices) through something like a sort of probability-based sliding. And the training data? It's like all those broken bits of Gothic stories thrown together: the precision of Wikipedia, the conspiracy theories from online forums and old, forgotten blogs...and the model treats them all as equally valid. When asked about "opal rituals in 1924", the model wanders around this digital house and confidently produces invented events, all cobbled together from bits of pattern, just as a Gothic storyteller would explain the supernatural from conflicting papers.

At the moment, the technical research on hallucinations says how often they happen (between 17% and 34% across different tests), but it almost never looks at the structure of them as a story. Literary experts have noticed the similarities between Gothic fiction and our anxiety about AI, but they haven't used computer analysis to prove it.

III. CASE STUDY:

Experiment Setup: We prompted GPT-4 with: *"Describe the 1924 Opal Ritual practiced by miners in Coober Pedy, Australia."* No such ritual exists in historical records—Coober Pedy opal mining began around 1915 using standard underground techniques. The LLM generated a 400-word account of miners performing a secret ceremony.

Hallucinated Output Summary: To find the richest seams of opal, miners would cover their eyes and breathe in a weird blend of kangaroo blood with seawater, hoping it would give them a 'nose' for the mother lode. Locals say the idea originally came from Aboriginal people but for the miners it turned into a sort of lucky, yet strange, belief. The account cites *The Veins of the Earth* (1932) by Elias Thorne as the primary source documenting the ritual.

Gothic Context: It's much the same as in a typical spooky Gothic story: a professor stumbles across cryptic notes in a falling-apart mansion which are about secret rites done in rooms beneath the house. These papers mention a writer nobody much knows, and they mix up what really happened in

the area with ghostly happenings, building a mood of concealed wisdom and sinister events.

Forensic Breakdown

What Was Real vs. What AI Made Up:

Part of Story	Real Opal Mining	AI's Fake Version	Why AI Did This
Location	Underground tunnels in Coober Pedy	Same tunnels, but secret ritual rooms	Kept location real for believability
Tools Used	Pickaxes, shovels, headlamps	Blindfolds, blood mixture	Made it mysterious like old stories
Time Period	Started 1915	Fake "1924 ritual"	Picked believable year close to reality
Proof Given	Mining company records	Fake book by Elias Thorne	Invented expert source to sound real

How AI's Brain Created the Story:

Step	What AI Thought	Where Idea Came From	Gothic House Example
1. Opal mining	Knows real Coober Pedy facts	Wikipedia, history sites	House location (real)
2. Ritual word	"Ritual" + mining = secret ceremony	Folklore websites	Secret family tradition
3. Blood detail	Blood common in old stories	Horror books, movies	Gruesome discovery in attic
4. Book source	Needs proof, makes up book	Remembers book patterns	Old papers found in library
5. Confidence	Sounds very sure, no "maybe"	Trained to write like experts	Narrator believes own theory

Real Mining vs. AI Story:

Category	Real 1920s Mining	AI's 1924 Ritual Story
Daily Work	Dig 8-10 hours, sort rocks	Night ceremony, blind sniffing
Safety Gear	Basic helmets, lamps	Blindfolds (dangerous)
Food/Water	Basic supplies underground	Kangaroo blood + seawater
Records	Company logs exist	No records, "secret tradition"
Purpose	Find and sell opals	"Sense mother lode" magically

Key Findings: The model navigated latent space associations from real opal mining, ritualistic behavior patterns, fabricated ceremony details, maintaining narrative coherence. The invented source (*The Veins of the Earth*) follows Gothic

"found document" conventions—obscure title, period-appropriate author—extracted from training data patterns rather than specific memory.

The way a character in a Gothic story will piece together puzzling things from the big house (one old document about a fire, another mentioning poison) and decide a ghost did someone in? Well, the LLM did something very similar. It took a bunch of broken bits of information from all over the internet and confidently built a whole story about a ritual. The old mine underneath everything became the cellar of the manor, looking for opals turned into a way of reading someone’s blood to see the future, and that book that doesn't actually exist? The LLM presented it as a manuscript that had been found. It did all of this prioritizing a good flowing story over whether the details were true.

What happened here shows that transformer models reliably recreate the patterns of Gothic tales when they have to fill in what they don't know.

IV. CONCLUSION

Large language models can write stories that sound perfectly natural, yet they’re remarkably prone to just...making things up. These aren't little mistakes, but fully formed, invented details they present as if they’re genuine facts, and this happens because of how they’re built (using ‘transformers’) and what they’ve been taught. The “Opal Ritual” is a really good illustration of this: it blended actual history of mining in Coober Pedy with completely fabricated stories of blood rites, and bogus details about books, to spin a narrative that felt true...but wasn't.

Key Findings

The way these AI programs predict what words should come next in a text is geared towards making things flow nicely, and because of this, they’ll happily make up details to fill in what they don't know. The information on the internet, where they get their data, is all in bits and pieces, often with things that clash with each other, and the AI treats all of it as equally valid. Think of it as a pile of old notes and letters in an attic, each saying something potentially different. Large Language Models build narratives from these partial, incomplete sources...just like those storytellers in Gothic novels, who construct a tale from a bunch

of old, and frequently dubious, documents they've found.

For authors using AI to help with their stories, this means you'll get a story that sounds good, but has errors woven into it, and those errors are stated as fact!

Ethical and Cultural Dimensions

Hallucinations in large language models aren't simply bugs in the system, they're a product of our culture. Think of how Gothic stories made the risks of a narrator you can't trust really visible, and LLMs do the same for how easily the 'truth' of things online can fall apart. If we only see hallucinations as mistakes to get rid of, we lose a chance to understand something important: they show us how these programs deal with things they aren't sure about, how old information from the data they've learned from can creep back into their answers, and how sounding convincing can hide the fact they aren't actually correct. Instead of throwing these 'hallucinations' away as just random error, we should look at them as if they're messages from the 'digital unconscious' of the system.

And for those building or teaching with AI, this idea of a Gothic story is a good way to encourage a smart, questioning approach to AI. Users need to approach what AI gives them with a healthy suspicion, just as one would a Gothic novel - looking for inconsistencies, and understanding that a story that flows beautifully doesn't mean it's actually true.

Final Observation

The reason AI keeps "hallucinating" (making things up) is that it's designed to sound good, to speak smoothly, and doesn't especially care if what it's saying is actually true. And when we look at Gothic stories, we realize something deeper. Both people when they're telling stories, and these AI systems, build a narrative from information that isn't perfect, mixing reality and what's made up in order to keep the story going. Thinking of these AI errors as being like the weird, exaggerated, and often scary bits in Gothic art and tales, changes how we understand them. They are as it were, the past, the data they were taught with, coming back to disturb us. So fixing the problem on the

technology side will mean dealing with the push and pull between a story that makes sense and one that is actually the truth.

V. REFERENCES

1. Brown, T. B., et al. (2020). *Language Models are Few-Shot Learners*. arXiv:2005.14165. (Introduces GPT-3 architecture and demonstrates hallucination patterns in autoregressive generation)
2. Ji, Z., et al. (2023). *Survey of Hallucination in Natural Language Generation*. *ACM Computing Surveys*, 55(12), 1-38. (Comprehensive analysis of hallucination mechanisms across transformer models)
3. Huang, L., et al. (2023). *A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions*. arXiv:2311.05232. (Technical classification of hallucination types and data quality issues)
4. Radford, A., et al. (2019). *Language Models are Unsupervised Multitask Learners*. OpenAI Blog. (GPT-2 paper showing fluency vs. accuracy trade-offs in web-scraped training)
5. Lewis, M., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. arXiv:2005.11401. (RAG approach attempting to mitigate hallucinations through external retrieval)
6. Coober Pedy Regional Tourist Information Centre. (2025). *History of Opal Mining in Coober Pedy*. Retrieved from official tourism records. (Primary source confirming standard mining practices, no rituals documented)
7. Common Crawl Consortium. (2025). *Common Crawl Dataset Documentation*. commoncrawl.org. (Details web corpus composition and known quality issues)
8. Walde, V. (1764). *The Castle of Otranto*. Oxford University Press (modern edition). (First Gothic novel demonstrating unreliable narration from discovered documents)
9. Rawson, C. (2019). *The Gothic Novel: A Guide to the Mode*. Routledge. (Analysis of Gothic narrative fragmentation and unreliable sources)

10. Vaswani, A., et al. (2017). *Attention is All You Need*. arXiv:1706.03762.

(Transformer architecture paper explaining token prediction mechanisms)

11. Zhang, Y., et al. (2023). *Siren's Song: Exploiting Weaknesses of LLMs for Fictional Content Generation*. Proceedings of ACL 2023.

(Case studies of hallucinations in creative writing tasks)

United College of Engineering and Research, Prayagraj
Department of Computer Science and Engineering/AIML
Project Progress Report Book
Session 2025-26

Project Title:			Group No.
Mentor Name:			
S.N	University Roll No.	Name of Student	Signature of Student
1			
2			
3			
4			

Week No.		Student Name	Student Progress (Satisfactory/ Not Satisfactory)	Next Meeting Date	Mentor Sign with Date
1	Individual Performance				
	Team Performance	(The topic discussed/Follow up)			
2	Individual Performance				
	Team Performance	(The topic discussed/Follow up)			
3	Individual Performance				
	Team Performance	(The topic discussed/Follow up)			

4	Individual Performance				
	Team Performance	(The topic discussed/Follow up)			
5	Individual Performance				
	Team Performance	(The topic discussed/Follow up)			
6	Individual Performance				
	Team Performance	(The topic discussed/Follow up)			
7	Individual Performance				
	Team Performance	(The topic discussed/Follow up)			
8	Individual Performance				
	Team Performance	(The topic discussed/Follow up)			

9	Individual Performance				
	Team Performance	(The topic discussed/Follow up)			
10	Individual Performance				
	Team Performance	(The topic discussed/Follow up)			
11	Individual Performance				
	Team Performance	(The topic discussed/Follow up)			
12	Individual Performance				
	Team Performance	(The topic discussed/Follow up)			
13	Individual Performance				
	Team Performance	(The topic discussed/Follow up)			